



รายงาน

การศึกษา Automated Test Case Generation Algorithms และ AI-Assisting Tools

กรณีศึกษา Whole Test Suite Generation (WTS), Coverage-guided Greybox

Fuzzing, Claude AI sonet 5-5 และ Gemini 3.8 flash

สำหรับการสร้างชุดทดสอบใน Java Projects ของ Defects4J

จัดทำโดย

นายพชรพงศ์ สาหลอน	รหัสนักศึกษา 673380414-7
นายนันทกร แสงวงจิตร	รหัสนักศึกษา 673380085-0
นายศุภวัฒน์ แสนเรียน	รหัสนักศึกษา 673380604-2

เสนอ

ผู้ช่วยศาสตราจารย์ ดร.จิตสุธา สุ่มเล็ก
วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353201 การประกันคุณภาพซอฟต์แวร์
Software Quality Assurance วิทยาลัยการคอมพิวเตอร์ สาขาวิทยาการคอมพิวเตอร์
มหาวิทยาลัยขอนแก่น ภาคการศึกษาที่ 1 ปีการศึกษา 2569

บทคัดย่อ

การสร้างชุดทดสอบเป็นกระบวนการสำคัญในการประกันคุณภาพซอฟต์แวร์ เนื่องจากช่วยตรวจสอบความถูกต้องของโปรแกรมและค้นหาข้อบกพร่องที่อาจเกิดขึ้น อย่างไรก็ตาม การสร้างชุดทดสอบด้วยมือสำหรับโครงการขนาดใหญ่อาจใช้เวลาและทรัพยากรจำนวนมาก โครงการนี้จึงมุ่งศึกษาแนวทางการสร้างชุดทดสอบ 4 แนวทาง ได้แก่ Whole Test Suite Generation (WTS), Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยใช้ Defects4J เป็นชุดข้อมูลสำหรับการทดลองกับโครงการภาษา Java

การทดลองแบ่งออกเป็น Automated Test Case Generation Algorithms และ AI-Assisting Tools โดย WTS ใช้ EvoSuite ในการสร้าง JUnit Test Suite และ Coverage-guided Greybox Fuzzing ใช้ Coverage Feedback ในการสร้างและคัดเลือกข้อมูลทดสอบ ส่วน Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ใช้ความสามารถของ Generative AI ผ่าน KKKU IntelSphere API เพื่อสร้างชุดทดสอบระดับ Unit Testing จากข้อมูลของ Bug และ Source Code โดยมีการสร้าง Prompt ตามข้อมูลของแต่ละ Bug และจัดเก็บผลลัพธ์เพื่อใช้ในการประเมิน

การประเมินผลพิจารณาจากความสามารถในการ Compile และ Execute, Test Coverage และ Fault Detection Rate (FDR) ผลการทดลองที่สามารถวัดผลได้แสดงให้เห็นว่า AI-Assisting Tools สามารถสร้างชุดทดสอบที่ตรวจจับข้อบกพร่องจริงได้ และสามารถนำไปใช้ร่วมกับกระบวนการประเมิน Test Coverage ได้ อย่างไรก็ตาม ผลการทดลองของแต่ละแนวทางมีขอบเขตและเงื่อนไขที่แตกต่างกัน รวมถึงข้อจำกัดด้าน API Quota และระยะเวลาในการทดลอง

ดังนั้น การเปรียบเทียบประสิทธิภาพจึงพิจารณาตามชุดข้อมูลและเงื่อนไขที่สามารถเปรียบเทียบกันได้ โดยไม่นำข้อมูลที่ยังไม่ได้ทดลองมาประมาณหรือสร้างขึ้น

คำสำคัญ การสร้างชุดทดสอบอัตโนมัติ, Automated Test Case Generation, AI-Assisting Tools, Whole Test Suite Generation, Coverage-guided Greybox Fuzzing, EvoSuite, Defects4J, Test Coverage, Fault Detection Rate

คำนำ

การทดสอบซอฟต์แวร์เป็นส่วนสำคัญของกระบวนการพัฒนาซอฟต์แวร์ เนื่องจากช่วยตรวจสอบความถูกต้องของการทำงานและค้นหาข้อบกพร่องที่อาจเกิดขึ้นก่อนนำซอฟต์แวร์ไปใช้งานจริง อย่างไรก็ตาม เมื่อซอฟต์แวร์มีขนาดใหญ่และมีความซับซ้อนมากขึ้น การสร้างชุดทดสอบด้วยมืออาจต้องใช้เวลาและทรัพยากรจำนวนมาก จึงเกิดการพัฒนาแนวทางการสร้างชุดทดสอบอัตโนมัติที่สามารถช่วยลดภาระของผู้ทดสอบและเพิ่มความครอบคลุมในการทดสอบ

ปัจจุบันมีแนวทางการสร้างชุดทดสอบหลายรูปแบบ ทั้งแนวทางที่อาศัยกระบวนการค้นหาและข้อมูลจากการทำงานของโปรแกรม เช่น Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing รวมถึงแนวทางที่นำความสามารถของ Generative AI มาใช้ในการวิเคราะห์ Source Code และสร้าง Test Code จากข้อมูลและคำสั่งภาษามนุษย์ เทคโนโลยีดังกล่าวมีความสามารถและข้อจำกัดที่แตกต่างกัน จึงมีความน่าสนใจในการศึกษาและเปรียบเทียบประสิทธิภาพภายใต้สภาพแวดล้อมการทดลองที่เหมาะสม

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353201 การประกันคุณภาพซอฟต์แวร์ (Software Quality Assurance) โดยจัดทำขึ้นเพื่อศึกษาแนวทางการสร้างชุดทดสอบ 4 แนวทาง ได้แก่ Whole Test Suite Generation (WTS), Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยใช้ Defects4J เป็นชุดข้อมูลสำหรับการทดลอง และพิจารณาผลการสร้างชุดทดสอบจาก Test Coverage และความสามารถในการตรวจจับข้อบกพร่อง รวมถึงข้อจำกัดที่เกิดขึ้นระหว่างกระบวนการทดลอง เนื้อหาในรายงานฉบับนี้ประกอบด้วยแนวคิดและทฤษฎีที่เกี่ยวข้อง วิธีดำเนินการทดลอง ผลการทดลอง การวิเคราะห์และอภิปรายผล ข้อจำกัดของการศึกษา ตลอดจนข้อสรุปและข้อเสนอแนะ เพื่อเป็นแนวทางสำหรับการศึกษการสร้างชุดทดสอบอัตโนมัติและการประยุกต์ใช้ AI-Assisting Tools ในการทดสอบซอฟต์แวร์ต่อไป

ผู้จัดทำหวังว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อการศึกษาและทำความเข้าใจแนวทางการสร้างชุดทดสอบอัตโนมัติ ตลอดจนการประยุกต์ใช้ AI-Assisting Tools ในงานด้านการประกันคุณภาพซอฟต์แวร์

คณะผู้จัดทำ

วันที่ 4 ตุลาคม 2569

สารบัญ

	หน้า
บทคัดย่อ	ก
คำนำ	ข
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญของโครงการ	
1.2 วัตถุประสงค์ของโครงการ	2
1.3 ขอบเขตของโครงการ	
บทที่ 2 แนวคิด ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	4
2.1 การสร้างกรณีทดสอบอัตโนมัติ	
2.2 AI-Assisting Tools สำหรับการสร้างชุดทดสอบ	6
2.3 การออกแบบ Prompt สำหรับการสร้าง Unit Test	7
2.4 ชุดข้อมูล Defects4J	9
2.5 ตัวชี้วัดในการประเมินการสร้างกรณีทดสอบ	
บทที่ 3 วิธีดำเนินการทดลอง	10
3.1 ภาพรวมการดำเนินการทดลอง	
3.2 การเตรียมข้อมูลการทดลอง	
3.3 การเตรียมอัลกอริทึมการสร้างกรณีทดสอบ	
3.4 การเตรียม Generative AI	12
3.5 การสร้างกรณีทดสอบ	13
3.6 การดำเนินการทดสอบ	
3.7 การวัด Test Coverage	14
3.8 การวัดความสามารถในการตรวจจับข้อบกพร่อง	
3.9 การเปรียบเทียบผลการทดลอง	15
บทที่ 4 ผลการทดลอง	16
4.1 ผลการทดลองการสร้างชุดทดสอบด้วย Whole Test Suite Generation	
4.2 ผลการทดลองการสร้างชุดทดสอบด้วย Coverage-guided Greybox Fuzzing	19
4.3 ผลการทดลอง Claude Sonnet 5.5	21
4.4 ผลการทดลอง Gemini 3.8 Flash	23
4.5 ผลการวัด Test Coverage	25
4.6 ผลการวัด Fault Detection	

บทที่ 5 วิเคราะห์และอภิปรายผล	27
5.1 การเปรียบเทียบ Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing	
5.2 การเปรียบเทียบ Claude Sonnet 5.5 และ Gemini 3.8 Flash	28
5.3 การเปรียบเทียบทั้ง 4 แนวทาง	30
5.4 ความสัมพันธ์ระหว่าง Test Coverage และ Fault Detection	32
5.5 การวิเคราะห์รูปแบบความล้มเหลว	34
5.6 อภิปรายผลการทดลอง	35
บทที่ 6 ข้อจำกัด ปัญหา และความสามารถในการทำซ้ำการทดลอง	37
6.1 ข้อจำกัดของการทดลอง	
6.2 ปัญหาและอุปสรรคในการดำเนินการทดลอง	38
6.3 ความสามารถในการทำซ้ำการทดลอง	39
บทที่ 7 สรุปผลและข้อเสนอแนะ	41
7.1 สรุปผลการศึกษา	
7.2 สรุปผลการเปรียบเทียบทั้ง 4 แนวทาง	42
7.3 ข้อจำกัดที่มีผลต่อการสรุปผล	43
7.4 ข้อเสนอแนะสำหรับการดำเนินงานต่อไป	
เอกสารอ้างอิง	45

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของโครงการ

การทดสอบซอฟต์แวร์เป็นขั้นตอนสำคัญของกระบวนการพัฒนาซอฟต์แวร์ เนื่องจากซอฟต์แวร์อาจยังมีข้อบกพร่องที่ซ่อนอยู่ในเส้นทางการทำงานบางรูปแบบ การทดสอบจึงมีหน้าที่ตรวจสอบพฤติกรรมและผลลัพธ์ของระบบ เพื่อช่วยลดความเสี่ยงของข้อผิดพลาดเมื่อซอฟต์แวร์ถูกนำไปใช้งานจริง

การสร้างกรณีทดสอบเป็นหนึ่งในขั้นตอนที่ใช้ทรัพยากรจำนวนมาก โดยเฉพาะโครงการที่มี Source Code จำนวนมากหรือมีเส้นทางการทำงานที่ซับซ้อน การสร้าง Test ด้วยมืออาจไม่สามารถครอบคลุมเส้นทางการทำงานที่เป็นไปได้ทั้งหมด จึงมีการพัฒนาแนวทางการสร้างกรณีทดสอบอัตโนมัติ เช่น Search-Based Software Testing และ Fuzzing เพื่อช่วยสร้างและปรับปรุงชุดทดสอบโดยอาศัยข้อมูลจากการทำงานของโปรแกรม

Whole Test Suite Generation (WTS) เป็นแนวทางหนึ่งของ Search-Based Software Testing ที่พิจารณา Test Suite เป็นชุดโดยรวม และใช้กระบวนการค้นหาเพื่อปรับปรุงชุดทดสอบให้บรรลุเป้าหมายด้าน Coverage ที่กำหนด โดย EvoSuite เป็นเครื่องมือที่นำแนวคิดดังกล่าวมาใช้กับภาษา Java ขณะที่ Coverage-guided Greybox Fuzzing ใช้ข้อมูล Coverage เป็น Feedback เพื่อคัดเลือก Input ที่สามารถสำรวจเส้นทางการทำงานใหม่และนำไปสร้าง Input เพิ่มเติม

นอกจากแนวทางดังกล่าวแล้ว ความก้าวหน้าของ Generative AI ทำให้เกิด AI-Assisting Tools ที่สามารถวิเคราะห์ Source Code และข้อมูลเกี่ยวกับ Bug เพื่อสร้าง Test Code ได้ การศึกษานี้จึงเลือกใช้ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash เป็น AI-Assisting Tools สำหรับสร้างชุดทดสอบระดับ Unit Testing อย่างไรก็ตาม Test Code ที่สร้างขึ้นอาจ Compile ไม่ผ่าน ไม่มี Assertion ที่เหมาะสม หรือไม่สามารถตรวจจับข้อบกพร่องได้ จึงจำเป็นต้องมีการประเมินผลอย่างเป็นระบบ

โครงการนี้เลือกใช้ Defects4J ซึ่งเป็นชุดข้อมูลข้อบกพร่องจริงจากโครงการภาษา Java และมีทั้ง Buggy Version และ Fixed Version ทำให้สามารถตรวจสอบความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องได้

ดังนั้น การศึกษานี้จึงมุ่งศึกษาและเปรียบเทียบแนวทางการสร้างชุดทดสอบ 4 แนวทาง ได้แก่ Whole Test Suite Generation (WTS), Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash โดยพิจารณา Test Coverage, Fault Detection Rate, จำนวน Test และระยะเวลาในการสร้างชุดทดสอบ รวมถึงข้อจำกัดที่เกิดขึ้นระหว่างการทดลอง

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อศึกษาแนวคิดและกระบวนการทำงานของ Automated Test Case Generation Algorithms และ AI-Assisting Tools สำหรับการสร้างชุดทดสอบระดับ Unit Testing
2. เพื่อศึกษาและประยุกต์ใช้ Whole Test Suite Generation (WTS) ผ่านเครื่องมือ EvoSuite สำหรับโครงการภาษา Java ใน Defects4J
3. เพื่อศึกษาและออกแบบ Coverage-guided Greybox Fuzzing สำหรับการสร้างข้อมูลทดสอบและชุดทดสอบ
4. เพื่อศึกษาและประยุกต์ใช้ AI-Assisting Tools ได้แก่ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ในการสร้างชุดทดสอบระดับ Unit Testing
5. เพื่อออกแบบ Prompt และกระบวนการสร้างชุดทดสอบที่สามารถดำเนินการอย่างเป็นระบบและสามารถทำซ้ำได้
6. เพื่อประเมินความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องจริงใน Defects4J และวัด Test Coverage ของชุดทดสอบที่สร้างขึ้น
7. เพื่อวิเคราะห์ปัจจัยที่ส่งผลต่อความสามารถในการสร้างชุดทดสอบและการตรวจจับข้อบกพร่อง
8. เพื่อเปรียบเทียบจุดเด่น ข้อจำกัด ประสิทธิภาพ และระยะเวลาในการสร้างชุดทดสอบของทั้ง 4 แนวทาง ได้แก่ Whole Test Suite Generation, Coverage-guided Greybox Fuzzing, Claude AI Sonnet 5.5 และ Gemini 3.8 Flash

1.3 ขอบเขตของโครงการ

1.3.1 ขอบเขตด้านชุดข้อมูล Defects4J การทดลองใช้ชุดข้อมูล Defects4J เวอร์ชัน 3.0.1 ซึ่งประกอบด้วยโครงการภาษา Java และข้อบกพร่องจริงจำนวน 854 Active Bugs จาก 17 โครงการ ได้แก่ Lang, Math, Chart, Closure, Time, Cli, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath และ Mockito การทดสอบแต่ละ Bug พิจารณา Buggy Version และ Fixed Version เพื่อประเมินความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่อง

1.3.2 ขอบเขตด้าน Automated Test Case Generation Algorithm การศึกษาด้าน Automated Test Case Generation Algorithms แบ่งออกเป็น 2 แนวทาง ได้แก่ Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing

1.3.2.1 Whole Test Suite Generation ใช้ EvoSuite เป็นเครื่องมือสำหรับสร้าง JUnit Test Suite โดยกำหนดเป้าหมายการค้นหา ได้แก่ Branch, Line, Exception และ Weak Mutation กำหนดงบประมาณ 120 วินาทีต่อ Bug ใช้ Random Seed เท่ากับ 1 และใช้คลาสที่ได้รับการแก้ไขของ Bug เป็น Target Class

1.3.2.2 Coverage-guided Greybox Fuzzing ศึกษาการสร้างข้อมูลทดสอบโดยใช้ Coverage Feedback ประกอบด้วยกระบวนการ Seed Selection, Mutation, Execution, Coverage Feedback และ Test Export โดยออกแบบให้สามารถประยุกต์ใช้กับการทดสอบระดับ Method หรือ Class ได้ การทดลองกำหนด Configuration ประกอบด้วย Budget 30, 60 และ 120 วินาทีต่อคลาส Seed Corpus หลายรูปแบบ และ Mutation Operators หลายระดับ โดยกำหนดการทำซ้ำ 5 รอบต่อ Configuration เพื่อใช้ในการวิเคราะห์ผลการทดลอง

1.3.3 ขอบเขตด้าน AI-Assisting Tools การทดลองด้าน AI-Assisting Tools ใช้ 2 เครื่องมือ ได้แก่

- 1 Claude AI Sonnet 5.5
- 2 Gemini 3.8 Flash

ทั้งสองเครื่องมือใช้งานผ่าน KKU IntelSphere API เพื่อสร้างชุดทดสอบระดับ Unit Testing สำหรับโครงการภาษา Java ใน Defects4J โดยใช้โปรแกรม Java ผ่าน Maven และ Java 17 สำหรับการเรียก API ส่วนการดำเนินการกับ Defects4J และการวัด Coverage ใช้ Java 11 และกำหนด max_tokens = 4096 สำหรับการเรียกใช้ API

สำหรับ Gemini มีการเพิ่ม EFFICIENCY NOTE ใน Prompt เพื่อควบคุมการใช้ Thinking Token หลังจากการทดลองเบื้องต้นพบว่าการใช้ Thinking Token อาจทำให้ Response ไม่สมบูรณ์ภายใต้ข้อจำกัดของ max_tokens โดยเงื่อนไขดังกล่าวถูกกำหนดเป็นส่วนหนึ่งของการทดลองและนำมาพิจารณาในการวิเคราะห์ผล

บทที่ 2

แนวคิด ทฤษฎี และเทคโนโลยีที่เกี่ยวข้อง

2.1 การสร้างกรณีทดสอบอัตโนมัติ

2.1.1 แนวคิดการสร้างกรณีทดสอบอัตโนมัติ

Automated Test Case Generation คือแนวทางที่ใช้กระบวนการอัตโนมัติในการสร้างกรณีทดสอบ แทนการออกแบบ Test Case ด้วยมือทั้งหมด โดยมีวิธีการและเป้าหมายที่แตกต่างกัน เช่น การเพิ่ม Code Coverage การค้นหาข้อบกพร่อง การสร้าง Input ที่มีลักษณะผิดปกติ และการตรวจสอบพฤติกรรมของโปรแกรมตามเงื่อนไขที่กำหนด

การประเมินประสิทธิภาพของการสร้างกรณีทดสอบอัตโนมัติไม่ควรพิจารณาจาก Coverage เพียงอย่างเดียว เนื่องจาก Coverage ที่สูงไม่ได้หมายความว่าชุดทดสอบจะสามารถตรวจจับข้อบกพร่องได้สูงเสมอไป ชุดทดสอบอาจสามารถเข้าถึง Statement หรือ Branch ของโปรแกรมได้ แต่ไม่มี Assertion ที่เหมาะสมในการตรวจสอบผลลัพธ์ ดังนั้น การประเมินจึงควรใช้ตัวชี้วัดหลายด้านร่วมกัน เช่น Test Coverage, Fault Detection Rate, จำนวน Test และระยะเวลาในการสร้างชุดทดสอบ เพื่อให้สามารถประเมินคุณภาพและประสิทธิภาพของชุดทดสอบได้อย่างเหมาะสม

2.1.2 Whole Test Suite Generation Whole Test Suite Generation (WTS) เป็นแนวทางหนึ่งของ Search-Based Software Testing ที่พิจารณา Test Suite ทั้งชุดเป็น Candidate Solution และใช้กระบวนการค้นหาเพื่อปรับปรุงคุณภาพของชุดทดสอบ โดยอาศัยค่า Fitness หรือ Coverage เป็นข้อมูลสำหรับประเมินชุดทดสอบ

กระบวนการทั่วไปของ WTS ประกอบด้วยการสร้าง Candidate Test Suite การนำชุดทดสอบไปทำงาน การประเมิน Fitness หรือ Coverage การคัดเลือกชุดทดสอบที่มีคุณภาพ และการสร้างหรือปรับปรุง Test เพิ่มเติม กระบวนการดังกล่าวจะดำเนินซ้ำจนกว่าจะบรรลุเงื่อนไขที่กำหนด จากนั้นจึงลด Test ที่ไม่จำเป็นเพื่อให้ได้ Final Test Suite ที่เหมาะสม

แนวทางดังกล่าวแตกต่างจากการสร้าง Test Case แยกตามเป้าหมาย เนื่องจาก Test หลายรายการภายใน Test Suite สามารถทำงานร่วมกันเพื่อค้นหาเส้นทางการทำงานและเพิ่ม Coverage ของโปรแกรมได้

2.1.3 เครื่องมือ EvoSuite EvoSuite เป็นเครื่องมือสำหรับการสร้าง JUnit Test Suite สำหรับโครงการภาษา Java โดยใช้แนวคิด Evolutionary Search และ Whole Test Suite Generation ในการค้นหาและปรับปรุงชุดทดสอบให้สามารถบรรลุเป้าหมายด้าน Coverage ที่กำหนด

ในการศึกษา EvoSuite ถูกนำมาใช้เป็นเครื่องมือสำหรับการทดลอง Whole Test Suite Generation โดยใช้ evosuite-current.jar ที่มาพร้อมกับสภาพแวดล้อมของ Defects4J เพื่อให้สามารถทำงานร่วมกับ Classpath ของโครงการที่ใช้ในการทดลองได้อย่างเหมาะสม

Configuration หลักที่ใช้ในการทดลองประกอบด้วย

ตารางที่ 1 ค่าการกำหนดสำหรับการทดสอบ Whole Test Suite Generation ด้วย EvoSuite

Parameter	ค่า
Defects4J	3.0.1
Total Budget	120 วินาที
Criterion	Branch, Line, Exception, Weak Mutation
Seed	1
Stopping Condition	MaxTime
Target	classes.modified
Project Classpath	defects4j export -p cp.compile

กระบวนการทำงานถูกจัดทำเป็น Shell Script เพื่อให้สามารถ Checkout Bug, Compile, Export Classpath, เรียก EvoSuite, บันทึกผล และทำความสะอาด Checkout ได้อย่างเป็นระบบ

2.1.4 Coverage-guided Greybox Fuzzing Coverage-guided Greybox Fuzzing เป็นแนวทางการสร้างข้อมูลทดสอบที่ใช้ Feedback จากการทำงานของโปรแกรมเพื่อควบคุมกระบวนการสร้าง Input โดยเริ่มจาก Seed Corpus แล้วนำ Seed ไปผ่านกระบวนการ Mutation เพื่อสร้าง Input ใหม่ จากนั้นนำ Input ที่ได้ไป Execute และตรวจสอบ Coverage หาก Input ใหม่สามารถทำให้เกิดเส้นทางการทำงานหรือ Coverage ที่ไม่เคยพบมาก่อน Input ดังกล่าวจะถูกเก็บไว้ใน Corpus และสามารถนำไปใช้เป็น Seed สำหรับการสร้าง Input ในรอบถัดไป

กระบวนการหลักของ Coverage-guided Greybox Fuzzing ประกอบด้วย

1. การเตรียม Seed Corpus
2. การเลือก Seed
3. การสร้าง Input ใหม่ด้วย Mutation
4. การนำ Input ไป Execute
5. การตรวจสอบ Coverage Feedback
6. การตรวจสอบ Crash หรือ Exception
7. การทำซ้ำจนถึงเวลาที่กำหนด
8. การนำข้อมูลทดสอบที่ได้ไปสร้างเป็น Unit Test

Mutation Operator ที่สามารถนำมาใช้ในการทดสอบ ได้แก่ Bit Flip, การเพิ่มหรือลดค่า, Boundary Value, Splice และ Dictionary-based Mutation โดยแต่ละรูปแบบมีลักษณะการเปลี่ยนแปลง Input ที่แตกต่างกัน และอาจส่งผลต่อความสามารถในการค้นหาเส้นทางการทำงานของโปรแกรม

เนื่องจากโครงงานนี้มุ่งเน้นการสร้างชุดทดสอบระดับ Unit Testing จึงจำเป็นต้องมี Fuzz Target หรือ Harness สำหรับกำหนด Method หรือ Class เป้าหมาย โดย Input ที่สร้างจาก Fuzzer จะถูกแปลงให้อยู่ในรูปแบบข้อมูลที่ Method สามารถรับได้ เช่น int, String, List, Object หรือค่าพารามิเตอร์ประเภทอื่น ๆ ในกรณีที่ Method ต้องใช้ Object ที่มีโครงสร้างซับซ้อน อาจจำเป็นต้องสร้าง Instance ผ่าน Constructor และเรียกใช้ Method ตามลำดับที่เหมาะสมเพื่อให้เกิดสถานะภายในที่แตกต่างกัน

สำหรับการตรวจสอบผลลัพธ์ของชุดทดสอบ เนื่องจาก Fuzzer ไม่สามารถทราบผลลัพธ์ที่ถูกต้องของทุก Input ได้โดยตรง โครงงานจึงกำหนดแนวทางให้ใช้ Fixed Version ของ Defects4J เป็น Regression Oracle โดยผลลัพธ์จาก Fixed Version สามารถใช้เป็น Expected Result สำหรับเปรียบเทียบกับผลลัพธ์จาก Buggy Version เพื่อประเมินความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องได้

2.2 AI-Assisting Tools สำหรับการสร้างชุดทดสอบ

2.2.1 แนวคิด Generative AI Generative AI เป็นเทคโนโลยีปัญญาประดิษฐ์ที่สามารถสร้างเนื้อหาใหม่จากข้อมูลและคำสั่งที่ได้รับ โดยสามารถนำมาประยุกต์ใช้ในการสร้าง Source Code และ Test Code จากคำอธิบายภาษาธรรมชาติได้

สำหรับการสร้างชุดทดสอบ Generative AI สามารถวิเคราะห์ข้อมูลที่เกี่ยวข้องกับโปรแกรม เช่น Source Code, API, Method และข้อมูลเกี่ยวกับ Bug แล้วนำข้อมูลดังกล่าวมาใช้ในการสร้าง Test Code ตามรูปแบบของ Test Framework เช่น JUnit การนำ Generative AI มาใช้ในลักษณะดังกล่าวจึงสามารถสนับสนุนกระบวนการสร้างชุดทดสอบระดับ Unit Testing ได้

2.2.2 Claude AI Sonnet 5.5 Claude AI Sonnet 5.5 เป็น AI-Assisting Tool ที่สามารถประมวลผลข้อมูลจาก Prompt และสร้าง Source Code หรือ Test Code ตามบริบทที่ได้รับ ในการศึกษาครั้งนี้ นำ Claude AI Sonnet 5.5 มาใช้สำหรับสร้างชุดทดสอบระดับ Unit Testing สำหรับโครงการภาษา Java ใน Defects4J โดยใช้ข้อมูลของ Bug, Source Code และบริบทที่เกี่ยวข้องเป็นข้อมูลประกอบการสร้าง Test Code

2.2.3 Gemini 3.8 Flash Gemini 3.8 Flash เป็น AI-Assisting Tool ที่สามารถวิเคราะห์ข้อมูลจาก Prompt และสร้าง Source Code หรือ Test Code ตามบริบทที่กำหนด ในการศึกษาครั้งนี้ นำ Gemini 3.8 Flash มาใช้สำหรับสร้างชุดทดสอบระดับ Unit Testing สำหรับโครงการภาษา Java ใน Defects4J โดยใช้ข้อมูลของ Bug, Source Code และบริบทที่เกี่ยวข้องเป็นข้อมูลประกอบการสร้าง Test Code

การใช้งาน Gemini ในการทดลองกำหนดข้อจำกัดด้านจำนวน Token ผ่านค่า max_tokens และมีการปรับ Prompt เพื่อควบคุมการใช้ Thinking Token ซึ่งรายละเอียดเกี่ยวกับรูปแบบ Prompt และเงื่อนไขการเรียกใช้ API จะอธิบายในบทที่ 3

2.3 การออกแบบ Prompt สำหรับการสร้าง Unit Test

การออกแบบ Prompt ในการทดลองมีวัตถุประสงค์เพื่อให้ AI-Assisting Tools สามารถสร้างชุดทดสอบระดับ Unit Test ที่เหมาะสมกับข้อบกพร่องแต่ละรายการใน Defects4J โดย Prompt ของแต่ละ Bug จะถูกสร้างจากข้อมูลของ Bug และโครงสร้างของโครงการนั้น ๆ แทนการใช้ Prompt แบบคงที่กับทุก Bug

เพื่อให้กระบวนการสร้าง Prompt มีความเป็นระบบและสามารถทำซ้ำได้ จึงพัฒนา PromptGenerator สำหรับสร้าง Prompt ของแต่ละ Bug โดยอัตโนมัติ และจัดเก็บ Prompt ที่ใช้จริงไว้ใน Repository เพื่อให้สามารถตรวจสอบข้อมูลที่ส่งให้ AI-Assisting Tools ได้

2.3.1 องค์ประกอบของ Prompt

Prompt ที่ใช้ในการทดลองประกอบด้วยข้อมูลสำคัญของ Bug ได้แก่

1. ข้อมูลของ Bug จาก Defects4J
2. ข้อมูลของ Trigger Test ที่เกี่ยวข้องกับ Bug
3. Source Code ที่เกี่ยวข้องกับ Bug
4. Diff ของ Source Code ระหว่าง Buggy Version และ Fixed Version
5. ข้อมูลของ Class หรือ Method ที่เกี่ยวข้องกับข้อบกพร่อง
6. ข้อกำหนดให้สร้างชุดทดสอบระดับ Unit Test
7. ข้อกำหนดให้ Test Code สามารถ Compile และ Execute ได้ในสภาพแวดล้อมของ Defects4J

Trigger Test ใช้เพื่อแสดงรูปแบบการเรียกใช้โปรแกรมและรูปแบบ Assertion ที่เกี่ยวข้องกับโครงการ ส่วน Source Code Diff ใช้เพื่อแสดงตำแหน่งและลักษณะของการเปลี่ยนแปลงที่เกี่ยวข้องกับข้อบกพร่อง ข้อมูลดังกล่าวช่วยให้ AI-Assisting Tools สามารถสร้าง Test Code โดยอาศัยบริบทของ Bug แต่ละรายการ

Prompt ของแต่ละ Bug ถูกจัดเก็บในรูปแบบ

```
Prompt/<Project>/<Project>-<N>/prompt.txt
```

การจัดเก็บในลักษณะดังกล่าวช่วยให้สามารถตรวจสอบย้อนหลังได้ว่าแต่ละ Bug ถูกนำข้อมูลใดมาใช้ในการสร้าง Prompt และช่วยสนับสนุนการทำซ้ำการทดลอง

2.3.2 การควบคุมความแตกต่างระหว่าง Claude และ Gemini

ในการทดลองใช้ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ผ่าน KIU IntelSphere API โดยกำหนด max_tokens = 4096 สำหรับการเรียกใช้ทั้งสองเครื่องมือ เนื่องจาก Gemini มีการใช้ Thinking Token ซึ่งอยู่ภายใต้ข้อจำกัดของ max_tokens จึงมีการเพิ่มข้อความ EFFICIENCY NOTE ใน Prompt ของ Gemini เพื่อควบคุมการใช้ Token และลดโอกาสที่ Response จะสิ้นสุดก่อนสร้าง Test Code ครบถ้วน โดยไม่ได้เพิ่มข้อความดังกล่าวใน Prompt ของ Claude

=== EFFICIENCY NOTE ===

Do not deliberate at length. Keep any internal reasoning very brief and write the final test code directly.

ความแตกต่างของ Prompt ดังกล่าวถือเป็นเงื่อนไขหนึ่งของการทดลอง และถูกนำมาพิจารณาเมื่อวิเคราะห์และอภิปรายผลการทดลองของ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash

2.3.3 การจัดเก็บ Prompt และ Response

Prompt ที่ใช้จริง รวมถึง Response ที่ได้รับจาก API ถูกจัดเก็บแยกตาม Project และ Bug เพื่อให้สามารถตรวจสอบกระบวนการทดลองย้อนหลังได้ Response ดิบจาก API ถูกเก็บไว้ใน

Result/<Project>/<Project>-<N>/raw_response.txt

ส่วน Test Code ที่ดึงออกจาก Response ถูกเก็บไว้ใน

TestCode/<Project>/<Project>-<N>/

ระบบจะตรวจสอบ stop_reason ก่อนนำ Response ไปใช้ หาก Response ไม่จบด้วย end_turn จะถือว่า Response ไม่สมบูรณ์และจะไม่สร้าง Test Code จาก Response ดังกล่าว แนวทางนี้ช่วยป้องกันไม่ให้นำ Test Code ที่ถูกตัดระหว่างการสร้างไปเข้าสู่ขั้นตอนการวัดผล และช่วยให้สามารถตรวจสอบสาเหตุของปัญหาจาก Response ดิบได้

2.4 ชุดข้อมูล Defects4J

Defects4J เวอร์ชัน 3.0.1 เป็นชุดข้อมูลที่ใช้ในการทดลองของโครงการ โดยประกอบด้วยโครงการภาษา Java และข้อบกพร่องจริงจำนวน 854 Active Bugs จาก 17 โครงการ ได้แก่ Lang, Math, Chart, Closure, Time, Cli, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath และ Mockito

Defects4J มี Buggy Version และ Fixed Version ซึ่งสามารถนำมาใช้ตรวจสอบว่าชุดทดสอบที่สร้างขึ้นสามารถแยกความแตกต่างระหว่างโปรแกรมที่มีข้อบกพร่องและโปรแกรมที่ได้รับการแก้ไขแล้วได้หรือไม่ การใช้ Defects4J ทำให้สามารถกำหนดสภาพแวดล้อมการทดลองที่มีข้อบกพร่องจริง และสามารถประเมิน Fault Detection ได้อย่างเป็นระบบ

2.5 ตัวชี้วัดในการประเมินการสร้างกรณีทดสอบ

2.5.1 Line Coverage

Line Coverage คือสัดส่วนของบรรทัด Source Code ที่ถูกเรียกใช้งานโดย Test เมื่อเทียบกับจำนวนบรรทัดทั้งหมดที่สามารถวัดได้

2.5.2 Branch Coverage

Branch Coverage ใช้ประเมินว่าการแตกแขนงของโปรแกรมได้รับการทดสอบมากน้อยเพียงใด

2.5.3 Condition Coverage

Condition Coverage พิจารณาการประเมินเงื่อนไขภายในส่วนที่มีการแตกแขนงของโปรแกรม

2.5.4 Fault Detection Rate

Fault Detection Rate ใช้ประเมินความสามารถของ Test ที่สร้างขึ้นในการตรวจจับข้อบกพร่องจริง โดยพิจารณาจาก Bug ที่ Test สามารถทำให้ Buggy Version ล้มเหลว แต่ Fixed Version ผ่านได้

บทที่ 3

วิธีดำเนินการทดลอง

3.1 ภาพรวมการดำเนินการทดลอง

การทดลองของโครงการแบ่งออกเป็น 4 แนวทาง ได้แก่ Automated Test Case Generation Algorithms จำนวน 2 แนวทาง และ AI-Assisting Tools จำนวน 2 แนวทาง โดยในส่วน AI-Assisting Tools ใช้ Claude Sonnet 5.5 และ Gemini 3.8 Flash ผ่าน KCU IntelSphere API ส่วนอัลกอริทึมสร้างกรณีทดสอบอัตโนมัติประกอบด้วย Whole Test Suite Generation (WTS) ด้วย EvoSuite และ Coverage-guided Fuzzing

แต่ละแนวทางจะสร้าง Test Suite หรือ Test Case จาก Bug ของ Defects4J จากนั้นนำ Test ที่ได้ไปประเมินกับ Buggy Version และ Fixed Version เพื่อพิจารณาความสามารถในการตรวจจับข้อบกพร่อง และวัด Test Coverage ตามเกณฑ์ที่กำหนด

3.2 การเตรียมข้อมูลการทดลอง

3.2.1 การเตรียมชุดข้อมูล Defects4J การทดลองใช้ชุดข้อมูล Defects4J เวอร์ชัน 3.0.1 โดยแต่ละ Bug จะต้องสามารถ Checkout Buggy Version และ Fixed Version เพื่อใช้ในการประเมิน Test ที่สร้างขึ้น

3.2.2 การเตรียม Buggy Version และ Fixed Version สำหรับการประเมิน Fault Detection จะนำ Test เดียวกันไปทดสอบกับทั้ง Buggy Version และ Fixed Version เพื่อพิจารณาว่า Test สามารถแยกความแตกต่างของโปรแกรมทั้งสองเวอร์ชันได้หรือไม่

3.2.3 การเตรียมสภาพแวดล้อมการทดลอง การทดลองมีการใช้ Java ต่างเวอร์ชันในแต่ละส่วน โดย API Client ใช้ Java 17 ขณะที่ Defects4J และ Coverage Workflow ใช้ Java 11 จึงจำเป็นต้องควบคุม Environment ให้ถูกต้อง

3.3 การเตรียมอัลกอริทึมการสร้างกรณีทดสอบ

3.3.1 การเตรียม Whole Test Suite Generation สำหรับ WTS ได้จัดทำกระบวนการทดลองด้วย EvoSuite และ Shell Script เพื่อให้สามารถดำเนินการกับ Bug ใน Defects4J อย่างเป็นระบบ ขั้นตอนประกอบด้วย

1. ตรวจสอบว่า Bug มีผลการทดลองอยู่แล้วหรือไม่
2. Checkout Buggy Version
3. Compile Project
4. ตรวจสอบ Modified Classes

5. Export Compile Classpath
6. เรียก EvoSuite
7. สร้าง Test Suite
8. บันทึกสถานะและเวลาการทำงาน
9. ทำความสะอาด Checkout
10. ดำเนินการ Bug ถัดไป

ระบบถูกออกแบบให้สามารถ Resume การทดลองได้ และรองรับการใช้งานร่วมกับ tmux เพื่อให้การทดลองจำนวนมากสามารถดำเนินต่อไปได้แม้ปิด Terminal

3.3.2 การตั้งค่า EvoSuite ในการทดลองนี้ไม่ได้ใช้ EvoSuite เป็นโปรแกรมแยกที่ดาวน์โหลดมาโดยตรง แต่ใช้ evosuite-current.jar ที่มากับสภาพแวดล้อม Defects4J เพื่อหลีกเลี่ยงปัญหาความเข้ากันได้ของ Classpath ที่พบจากการใช้ EvoSuite รุ่นอื่น

Configuration ของการทดลอง Round 1 กำหนดงบเวลา 120 วินาทีต่อ Bug ใช้ Random Seed เท่ากับ 1 และใช้คลาสที่ได้รับการแก้ไขของ Bug เป็น Target Class

เป้าหมายการค้นหา ได้แก่ Branch, Line, Exception และ Weak Mutation

3.3.3 การเตรียม Coverage-guided Greybox Fuzzing ในส่วนของ Fuzzing มีการศึกษาและออกแบบกระบวนการสำหรับนำ Coverage-guided Greybox Fuzzing มาประยุกต์ใช้กับ Unit Testing โดยมีองค์ประกอบสำคัญ ได้แก่

- Seed Corpus
- Seed Selection
- Mutation
- Execution
- Coverage Feedback
- Crash Detection
- Test Minimization
- JUnit Test Export

3.3.4 การกำหนด Configuration ของ Fuzzing การออกแบบ Configuration เสนอให้ทดลองหลายระดับเพื่อศึกษาผลของ Budget และรูปแบบการสร้าง Input ได้แก่

Parameter	Configuration
Budget	30, 60, 120 วินาที
Seed Corpus	Empty / Boundary / Existing Tests
Mutation	Basic / Basic + Splice + Dictionary
Power Schedule	Uniform / Coverage-new
Repetition	5 ครั้งต่อ Configuration
Random Seed	กำหนดคงที่

ตารางที่ 2 การกำหนดค่า Configuration สำหรับการทดลอง Coverage-guided Greybox Fuzzing

ตัวชี้วัดที่เสนอประกอบด้วย Line Coverage, Branch Coverage, Fault Detection Rate, Mutation Score, จำนวน Test, ขนาด Test Suite, เวลาในการสร้าง Test, เวลาที่ Coverage ถึงจุดอิ่มตัว และ Flaky Rate

3.4 การเตรียม Generative AI

3.4.1 การเรียกใช้งาน KGU IntelSphere API การเรียก API ใช้โปรแกรม Java ผ่าน Maven และ Java 17 โดยกำหนด max_tokens = 4096 สำหรับทั้งสองโมเดล

3.4.2 การเตรียม Claude Sonnet 5.5 Claude Sonnet 5.5 ถูกนำมาใช้สร้าง Unit Test ผ่าน KGU IntelSphere API โดยใช้ Prompt ที่สร้างจากข้อมูลของแต่ละ Bug

3.4.3 การเตรียม Gemini 3.8 Flash Gemini 3.8 Flash ถูกนำมาใช้สร้าง Unit Test ผ่าน KGU IntelSphere API เช่นเดียวกับ Claude แต่มีการเพิ่ม EFFICIENCY NOTE เพื่อควบคุมการใช้ Thinking Token

3.5 การสร้างกรณีทดสอบ

3.5.1 การสร้าง Test ด้วย WTS EvoSuite ถูกใช้เพื่อสร้าง JUnit Test Suite โดยใช้กระบวนการ Search-Based Software Testing และ Whole Test Suite Generation

3.5.2 การสร้าง Test ด้วย Fuzzing Fuzzing เริ่มต้นจาก Seed Corpus จากนั้นเลือก Seed และสร้าง Input ใหม่ด้วย Mutation Operators เมื่อนำ Input ไป Execute แล้วพบ Coverage ใหม่ Input ดังกล่าวจะถูกเก็บกลับเข้า Corpus และสามารถนำมาใช้ในรอบต่อไป

3.5.3 การสร้าง Test ด้วย Claude สำหรับส่วน Generative AI ระบบจะสร้าง Prompt จากข้อมูลของแต่ละ Bug แล้วส่งไปยังโมเดลผ่าน KGU IntelSphere API โดยกำหนด max_tokens = 4096 ทั้งสองโมเดล

3.5.4 การสร้าง Test ด้วย Gemini Gemini ใช้กระบวนการเดียวกับ Claude แต่ Prompt มี EFFICIENCY NOTE เพิ่มเติมเพื่อควบคุมการใช้ Token การสร้าง Test จะเป็นการทดลองแบบหนึ่งครั้งต่อ Bug โดยไม่มีการแก้ไข Test ที่โมเดลสร้างขึ้นและไม่มีการเรียก API ซ้ำเพื่อสร้าง Test ใหม่เมื่อผลลัพธ์ไม่เป็นไปตามที่ต้องการ

3.6 การดำเนินการทดสอบ

ขั้นตอนการทำงานของส่วน Generative AI สามารถสรุปได้ดังนี้

1. เตรียม Buggy Version และ Fixed Version ของ Bug จาก Defects4J
2. ดึงข้อมูล Bug, Trigger Test และ Diff ของ Source Code
3. สร้าง Prompt ด้วย PromptGenerator
4. ส่ง Prompt ไปยัง Generative AI ผ่าน KGU IntelSphere API
5. บันทึก Response ที่ได้รับ
6. ดึง Test Code จาก Response
7. ตรวจสอบและเตรียม Test สำหรับการทดลอง
8. Compile และรัน Test บน Buggy Version
9. Compile และรัน Test บน Fixed Version
10. พิจารณาว่า Test สามารถตรวจจับ Bug ได้หรือไม่
11. วัด Line Coverage และ Condition Coverage
12. บันทึกผลการทดลองลงในไฟล์ CSV และสรุปผลของแต่ละโครงการ

3.7 การวัด Test Coverage

3.7.1 การวัด Line Coverage Line Coverage คือสัดส่วนของบรรทัด Source Code ที่ถูกเรียกใช้งานโดย Test เมื่อเทียบกับจำนวนบรรทัดทั้งหมดที่สามารถวัดได้ โดยในการทดลองนี้วัด Coverage บน Fixed Version และพิจารณาเฉพาะคลาสที่ได้รับการแก้ไขจาก Bug นั้น สามารถคำนวณได้ดังนี้

$$\text{Line Coverage} = \text{Covered Lines} / \text{Total Lines} \times 100$$

ค่า Line Coverage ที่ได้จึงไม่ได้หมายถึง Coverage ของทั้งโครงการ แต่เป็น Coverage ของคลาสที่เกี่ยวข้องกับ Bug ที่กำลังประเมินเท่านั้น

3.7.2 การวัด Branch Coverage Branch Coverage ใช้ประเมินว่าการแตกแขนงของโปรแกรมได้รับการทดสอบมากน้อยเพียงใด

3.7.3 การวัด Condition Coverage Condition Coverage พิจารณาการประเมินเงื่อนไขภายในส่วนที่มีการแตกแขนงของโปรแกรม

ในการทดลอง Generative AI ใช้ Line Coverage และ Condition Coverage เป็นตัวชี้วัดหลักเพิ่มเติมจาก Fault Detection Rate เพื่อแสดงให้เห็นว่า Test ที่สร้างขึ้นสามารถเข้าถึงส่วนต่าง ๆ ของ Source Code ได้มากน้อยเพียงใด

อย่างไรก็ตาม Coverage ไม่ได้หมายความว่า Test สามารถตรวจจับ Bug ได้เสมอไป เนื่องจาก Test อาจสามารถ Execute Code ได้เป็นจำนวนมากแต่ไม่มี Assertion หรือไม่มีเงื่อนไขที่สามารถแสดงความแตกต่างระหว่าง Buggy Version และ Fixed Version ได้ ดังนั้นจึงใช้ FDR ร่วมกับ Coverage ในการประเมิน

3.8 การวัดความสามารถในการตรวจจับข้อบกพร่อง

3.8.1 เกณฑ์การตัดสิน Fault Detection หลักการสำคัญของการประเมินคือ Test ที่สร้างขึ้นต้องสามารถแยกความแตกต่างระหว่าง Buggy Version และ Fixed Version ได้ โดยกำหนดว่า Test สามารถตรวจจับ Bug ได้เมื่อเป็นไปตามเงื่อนไขทั้ง 4 ข้อ ได้แก่

1. Compile บน Buggy Version ได้
2. Compile บน Fixed Version ได้
3. Test ล้มเหลวบน Buggy Version
4. Test ผ่านบน Fixed Version

การกำหนดเงื่อนไขทั้ง 4 ข้อช่วยลดความเสี่ยงในการนับ Test ที่ล้มเหลวจากสาเหตุอื่น เช่น Test เขียนผิด การเรียก API ไม่ถูกต้อง หรือปัญหาจาก Environment เป็น Test ที่สามารถตรวจจับ Bug ได้

3.8.2 การคำนวณ Fault Detection Rate กำหนดอัตราการตรวจจับข้อบกพร่อง (Fault Detection Rate: FDR) ดังนี้

$$FDR = \text{จำนวน Bug ที่ตรวจจับได้} / \text{จำนวน Bug ที่สามารถวัดผลได้} \times 100$$

3.8.3 การจัดกลุ่มผลการทดลอง ในการจัดกลุ่มผลการทดลอง แบ่งออกเป็น 4 กลุ่มหลัก ได้แก่

ตารางที่ 3 การจัดกลุ่มผลการทดลองตามความสามารถในการตรวจจับข้อบกพร่อง

กลุ่ม	เงื่อนไข
Detected	Compile ผ่านทั้งสอง Version, Test ล้มเหลวบน Buggy และผ่านบน Fixed
Not Detected	Test ผ่านทั้งสอง Version, ล้มเหลวทั้งสอง Version หรือ Compile ไม่ผ่านเนื่องจากปัญหาของ Test
วัดไม่ได้	ปัญหาเกิดจาก Helper, Script หรือ Environment ที่ไม่ใช่ความผิดของ Test
ตอบไม่จบ	โมเดลไม่สามารถสร้าง Test Code ได้เนื่องจาก Response สิ้นสุดที่ max_tokens

กรณี “วัดไม่ได้” และ “ตอบไม่จบ” จะไม่ถูกนำไปรวมในตัวหารของ FDR หลัก แต่จะแสดงจำนวนแยกต่างหาก เพื่อให้สามารถพิจารณาผลกระทบของปัญหาดังกล่าวได้อย่างชัดเจน นอกจากนี้สามารถนำทั้งสองกลุ่มมานับเป็นความล้มเหลวเพิ่มเติมในการวิเคราะห์ความไว (Sensitivity Analysis) ได้

3.9 การเปรียบเทียบผลการทดลอง

การเปรียบเทียบแนวทางทั้ง 4 วิธีจะพิจารณาจากลักษณะของวิธีการสร้าง Test และตัวชี้วัดที่กำหนด ได้แก่ Coverage, Fault Detection, จำนวน Test และเวลาในการสร้าง Test อย่างไรก็ตาม การเปรียบเทียบเชิงตัวเลขจะดำเนินการเฉพาะกรณีที่มีชุดข้อมูลและเงื่อนไขการทดลองที่สามารถเปรียบเทียบกันได้

บทที่ 4

ผลการทดลอง

4.1 ผลการทดลองการสร้างชุดทดสอบด้วย Whole Test Suite Generation

ตารางที่ 4 ผลการรันรอบที่ 1 Budget 120 วินาที

Project	จำนวน Bug ใน Defects4J	จำนวนคลาสที่รันสำเร็จ (ok)	เวลาเฉลี่ยต่อคลาส (วินาที)	Line Coverage เฉลี่ย (%)
Chart	26	30	82.3	82.50%
Cli	40	51	50.2	78.40%
Closure	176	226	68.8	75.20%
Codec	18	29	44.1	83.10%
Collections	28	30	77.2	77.90%
Compress	47	60	65.8	68.40%
Csv	16	19	77.1	81.20%
Gson	18	21	59.9	69.50%
JacksonCore	26	35	71.3	65.40%
JacksonDatabind	112	157	60.6	61.80%
JacksonXml	6	6	74.5	84.10%
Jsoup	93	126	55.7	76.80%
JXPath	22	35	50.1	74.30%
Lang	65	61	94.2	81.50%
Math	106	122	81.6	72.10%
Mockito	38	46	47.2	52.40%
Time	27	34	66.7	80.30%

รวม / ค่าเฉลี่ย ทั้งสิ้น	854	1,088	66.4	74.50%
-----------------------------	-----	-------	------	--------

จากการทดลองรัน EvoSuite ในรอบที่ 1 ด้วยเวลา 120 วินาทีต่อข้อบกพร่อง จากชุดข้อมูลมาตรฐาน Defects4J ทั้งหมด 854 บั๊ก ระบบสามารถสังเคราะห์และสร้างชุดทดสอบสำหรับคลาสเป้าหมายที่ผ่านสถานะสมบูรณ์ (ok) ได้รวมทั้งสิ้น 1,088 คลาส ใช้เวลาในการประมวลผลเฉลี่ยภาพรวมอยู่ที่ 66.4 วินาทีต่อคลาส และมีค่าเฉลี่ยความครอบคลุมของซอร์สโค้ด (Line Coverage) ภาพรวมอยู่ที่ 74.5%

จากการวิเคราะห์ผลลัพธ์ในตาราง พบว่าจำนวนคลาสที่รันสำเร็จ (ok) ในหลายโครงการมีจำนวนสูงกว่าจำนวนข้อบกพร่องตั้งต้นใน Defects4J (เช่น โครงการ Chart มี 26 บั๊ก แต่สร้างเทสสำเร็จ 30 คลาส หรือ Closure มี 176 บั๊ก แต่สำเร็จถึง 226 คลาส)

ทั้งนี้เนื่องมาจากลักษณะการแก้ไขข้อบกพร่อง (Bug Fix) ในชุดข้อมูล Defects4J 1 รายการ นักพัฒนาอาจมีการกระจายแก้ไขซอร์สโค้ดมากกว่า 1 คลาส (Multi-class Fixes) ซึ่งเครื่องมืออัตโนมัติต้องดำเนินการสร้างชุดทดสอบครอบคลุมทุกคลาสเป้าหมายที่เกี่ยวข้องทั้งหมด

ด้านความครอบคลุม (Coverage) โครงการที่เครื่องมือสร้างเทสแล้วได้ค่า Line Coverage สูงที่สุดในรอบนี้คือ JacksonXml (84.1%) และ Codec (83.1%) ตามลำดับ แสดงว่าโครงสร้างโค้ดมีความซับซ้อนน้อยกว่าหรือเอื้อต่อการสร้าง Assertion ของ EvoSuite ได้ดี

ด้านระยะเวลา (Duration): โครงการที่ใช้เวลาเฉลี่ยสูงสุดคือ Lang (94.2 วินาที/คลาส) เนื่องจากมีขนาดโค้ดและเมธอดที่ซับซ้อน ทำให้เข้าใกล้ขอบเขต Budget ที่ตั้งไว้ (120 วินาที) มากที่สุด

ตารางที่ 5 ผลการรันรอบที่ 2 Budget 60 วินาที

Project	จำนวน Bug ใน Defects4J	จำนวนคลาสที่ บันทึกผลสำเร็จ (ok)	เวลาเฉลี่ยต่อ คลาส (วินาที)	Line Coverage เฉลี่ย (%)
Chart	26	25	48.2	78.00%
Cli	40	34	40.5	74.50%
Closure	176	111	38.4	35.40%
Codec	18	22	25.8	78.20%
Collections	28	28	41.1	74.80%

Compress	47	47	38.6	55.60%
Csv	16	17	41	80.20%
Gson	18	20	35.2	62.30%
JacksonCore	26	33	45.3	58.40%
JacksonDatabind	112	77	35.5	60.50%
JacksonXml	6	6	41.5	N/A*
Jsoup	93	75	35.1	72.10%
JXPath	22	21	32.4	51.20%
Lang	65	61	65.2	80.10%
Math	106	102	55.4	70.50%
Mockito	38	35	25.5	48.50%
Time	27	23	43.7	82.00%
รวม / ค่าเฉลี่ย ภาพรวม	854	737	41.7	64.60%

จากการทดลองรัน EvoSuite ในรอบที่ 2 ด้วยงบประมาณ 60 วินาทีต่อข้อบกพร่อง พบว่าจากเป้าหมายข้อบกพร่องตั้งต้น 854 รายการ เครื่องมือสามารถสร้างไฟล์ซอร์สโค้ดชุดทดสอบได้สำเร็จถึง 1,019 คลาส อย่างไรก็ตาม มีคลาสที่สามารถดำเนินการรวบรวมและบันทึกผลลัพธ์ทางสถิติลงไฟล์ข้อมูลสำเร็จเพียง 737 คลาส โดยใช้เวลาในการประมวลผลเฉลี่ยภาพรวมลดลงเหลือ 41.7 วินาทีต่อคลาส และมีค่าเฉลี่ยความครอบคลุมของซอร์สโค้ด (Line Coverage) ภาพรวมอยู่ที่ 64.6%

การที่จำนวนคลาสที่บันทึกผลสำเร็จมีจำนวนน้อยกว่าไฟล์ชุดทดสอบที่ถูกสร้างขึ้นจริง เกิดจากปรากฏการณ์ข้อจำกัดด้านเวลา (Timeout Effect) เมื่อระยะเวลาถูกจำกัดไว้ที่ 60 วินาที สำหรับโครงการหรือคลาสที่มีความซับซ้อนสูง ระบบจะทำการตัดการประมวลผล (Kill Process) ทันทีเมื่อหมดเวลา ส่งผลให้เครื่องมือสามารถสร้างไฟล์ชุดทดสอบ (.java) ได้ทัน แต่ไม่สามารถดำเนินการในขั้นตอนสุดท้ายคือการคำนวณและบันทึกค่าความครอบคลุมลงในไฟล์สถิติได้ทันเวลา

เมื่อเปรียบเทียบกับผลการทดลองในรอบที่ 1 พบความเชื่อมโยงเชิงแลกเปลี่ยน (Trade-off) อย่างมีนัยสำคัญ การลดงบประมาณลง 50% ช่วยให้เวลาเฉลี่ยในการรันต่อคลาสลดลงจาก 66.4 วินาที เหลือ 41.7

วินาที แต่ต้องแลกมาด้วยค่าความครอบคลุมภาพรวมที่ลดลงจาก 74.5% เหลือ 64.6% โดยเฉพาะอย่างยิ่งในโครงการที่มีขนาดใหญ่และซับซ้อน เช่น Closure ที่ค่าความครอบคลุมลดลงอย่างรวดเร็ว (เหลือเพียง 35.4%) แสดงให้เห็นว่าระยะเวลา 60 วินาทีอาจไม่เพียงพอสำหรับอัลกอริทึมในการค้นหาและเจาะลึกโครงสร้างเงื่อนไขที่ซับซ้อนได้อย่างสมบูรณ์

4.2 ผลการทดลองการสร้างชุดทดสอบด้วย Coverage-guided Greybox Fuzzing

การทดลองนี้ประเมินผลการทำงานของ Algorithm 2 จากข้อมูลการรันใน Round 1 และ Round 2 โดยพิจารณาจำนวนรายการที่ปรากฏในไฟล์สรุป สถานะการรัน เวลาเฉลี่ยต่อรายการ และ Line Coverage เฉลี่ยแยกตามโปรเจกต์ เวลาเฉลี่ยคำนวณจากรายการที่มีเวลารันมากกว่า 0 วินาที โดยรวมทุกสถานะ ส่วน Line Coverage เฉลี่ยคำนวณจากรายการที่มีข้อมูล target class ในรายงาน JaCoCo รวมรายการที่มีสถานะ CRASH หากยังมีข้อมูล coverage ให้นำมาคำนวณด้วย

ตารางที่ 6 ผลการรันรอบที่ 1

Project	รายการใน summary.csv	PASS	เวลาเฉลี่ยต่อรายการ (วินาที)	Line Coverage เฉลี่ย (%)
Cli	39	32	56.34	62.30
Closure	132	0	63.93	0.00
Codec	18	2	50.33	66.13
Collections	28	0	54.54	44.54
Compress	47	28	56.68	58.29
Csv	16	0	57.12	47.62
Gson	18	0	63.56	50.54
JacksonCore	26	6	59.08	74.41
JacksonDatabind	112	0	56.37	0.00
JacksonXml	6	0	55.17	0.00
Jsoup	93	0	54.56	9.84
JXPath	22	0	59.14	31.11

Lang	65	0	54.36	19.32
Lang_f	65	0	ไม่มีข้อมูล	ไม่มีข้อมูล
Math	105	0	56.08	73.04
Mockito	37	0	64.62	17.62
Time	27	0	66.81	8.62
รวม / ค่าเฉลี่ย	856	68	58.11	23.66

รอบที่ 1 มีรายการใน summary.csv จำนวน 856 รายการ ในจำนวนนี้ 781 รายการมีเวลารันจริง และ 68 รายการมีสถานะ PASS เวลาเฉลี่ยจากรายการที่มีเวลารันอยู่ที่ 58.11 วินาทีต่อรายการ ส่วน Line Coverage เฉลี่ยอยู่ที่ 23.66% จาก 660 รายการที่มีข้อมูล coverage เพียงพอสำหรับคำนวณ

ตารางที่ 7 ผลการรันรอบที่ 2

Project	รายการใน summary.csv	PASS	เวลาเฉลี่ยต่อรายการ (วินาที)	Line Coverage เฉลี่ย (%)
Cli	39	32	57.87	62.30
Closure	133	0	58.38	0.00
Codec	18	1	51.67	66.80
Collections	28	0	54.86	35.89
Compress	47	28	53.30	58.39
Csv	16	0	50.00	50.05
Gson	18	0	53.17	52.40
JacksonCore	26	7	56.27	74.22
JacksonDatabind	110	0	52.18	0.00
JacksonXml	6	0	49.83	0.00
Jsoup	92	0	52.32	9.84
JXPath	22	0	50.55	29.81

Lang	65	0	51.61	19.65
Lang_f	65	0	ไม่มีข้อมูล	ไม่มีข้อมูล
Math	106	0	53.85	72.62
Mockito	38	0	59.08	17.00
Time	27	0	58.77	8.02
รวม / ค่าเฉลี่ย	856	68	54.45	23.62

รอบที่ 2 มีรายการใน summary.csv จำนวน 856 รายการ มี 781 รายการที่ใช้เวลารันจริง และ 68 รายการมีสถานะ PASS เวลาเฉลี่ยอยู่ที่ 54.45 วินาทีต่อรายการ ส่วน Line Coverage เฉลี่ยอยู่ที่ 23.62% จาก 659 รายการที่มีข้อมูล coverage เพียงพอ

เมื่อเปรียบเทียบสองรอบ เวลาเฉลี่ยลดลงจาก 58.11 วินาทีใน Round 1 เป็น 54.45 วินาทีใน Round 2 ขณะที่ Line Coverage เฉลี่ยใกล้เคียงกัน คือ 23.66% และ 23.62% ตามลำดับ ค่า Line Coverage ในตารางนี้รวมรายการที่มีข้อมูล coverage แม้สถานะจะเป็น CRASH ด้วย ส่วนรายการที่ไม่มีรายงานหรือไม่มีข้อมูล target class จะไม่ถูกนับเป็น 0% และไม่นำมาคำนวณค่าเฉลี่ย

4.3 ผลการทดลอง Claude Sonnet 5.5

Claude Sonnet 5.5 ถูกนำมาใช้ในการสร้างชุดทดสอบระดับ Unit Test สำหรับโครงการภาษา Java ในชุดข้อมูล Defects4J โดยการทดลองครอบคลุมทั้งหมด 9 โครงการ ได้แก่ Cli, Closure, Codec, Csv, Gson, JacksonXml, JXPath, Math และ Time ผลการประเมินพิจารณาจากความสามารถในการตรวจจับข้อบกพร่องจริงของแต่ละโครงการ และการวัด Code Coverage ของชุดทดสอบที่สร้างขึ้น โดยใช้ตัวชี้วัด Fault Detection Rate (FDR), Line Coverage และ Condition Coverage ผลการทดลองแสดงดังตารางที่ 8

ตารางที่ 8 ผลการตรวจจับข้อบกพร่องและ Code Coverage จากชุดทดสอบที่สร้างโดย Claude Sonnet

5.5

Project	จำนวน Bug	Bug ที่ตรวจจับได้	Bug ที่ตรวจจับไม่ได้	FDR	Line Coverage	Condition Coverage
Cli	38	36	2	94.74%	42.41%	30.48%
Closure	174	88	82	51.76%	36.07%	24.77%
Codec	18	16	2	88.89%	42.18%	33.83%
Csv	16	13	3	81.25%	33.66%	19.85%
Gson	18	17	1	94.44%	38.57%	29.67%
JacksonXml	6	5	1	83.33%	27.10%	19.65%
JXPath	22	9	2	81.82%*	44.80%	33.71%
Math	106	97	7	93.27%	41.42%	31.13%
Time	26	25	0	100.00%*	20.14%	14.94%
ค่าเฉลี่ย	—	—	—	85.50%	36.26%	26.45%

จากผลการทดลอง พบว่า Claude Sonnet 5.5 สามารถสร้างชุดทดสอบที่ตรวจจับข้อบกพร่องของ Defects4J ได้ในทุกโครงการที่นำมาทดลอง โดยเมื่อพิจารณาจากข้อบกพร่องทั้งหมด 424 รายการ สามารถตรวจจับได้ 306 รายการ ขณะที่ Bug ที่ไม่สามารถตรวจจับได้ 100 รายการ และมี Bug อีก 18 รายการที่ไม่สามารถนำมาประเมินผลได้ เนื่องจากข้อจำกัดในการทดสอบหรือสภาพแวดล้อมของโครงการ เมื่อคำนวณ FDR โดยพิจารณาเฉพาะ Bug ที่สามารถนำมาวัดผลได้จำนวน 406 รายการ พบว่า Claude Sonnet 5.5 สามารถตรวจจับ Bug ได้ 306 รายการ คิดเป็น FDR รวม 75.37% ส่วนค่าเฉลี่ย FDR ของทั้ง 9 โครงการอยู่ที่ 85.50%

เมื่อพิจารณาเป็นรายโครงการ พบว่า Time มีค่า FDR สูงที่สุดที่ 100.00% โดยสามารถตรวจจับ Bug ที่นำมาวัดผลได้ทั้งหมด 25 รายการ รองลงมาคือ Cli ที่มีค่า FDR 94.74% และ Gson ที่มีค่า FDR 94.44% ขณะที่ Closure มีค่า FDR ต่ำที่สุดที่ 51.76% โดยสามารถตรวจจับ Bug ได้ 88 จาก 170 รายการที่สามารถนำมาวัดผลได้

นอกจากการประเมินความสามารถในการตรวจจับข้อบกพร่องแล้ว ยังได้วัด Code Coverage ของชุดทดสอบที่สร้างขึ้น โดยพิจารณาจาก Line Coverage และ Condition Coverage จากผลการทดลอง พบว่า ค่าเฉลี่ย Line Coverage ของทั้ง 9 โครงการอยู่ที่ 36.26% และค่าเฉลี่ย Condition Coverage อยู่ที่ 26.45%

ผลการทดลองโดยรวมแสดงให้เห็นว่า Claude Sonnet 5.5 สามารถสร้างชุดทดสอบระดับ Unit Test ที่มีความสามารถในการตรวจจับข้อบกพร่องจริงและสามารถครอบคลุมส่วนต่าง ๆ ของ Source Code ได้ในระดับหนึ่ง อย่างไรก็ตาม ผลลัพธ์ของแต่ละโครงการมีความแตกต่างกัน ทั้งในด้าน FDR และ Code Coverage ซึ่งอาจสัมพันธ์กับจำนวนและลักษณะของ Bug โครงสร้างของ Source Code และเงื่อนไขของแต่ละโครงการ ดังนั้นการประเมินประสิทธิภาพของ Claude Sonnet 5.5 จึงควรพิจารณาหลายตัวชี้วัดร่วมกัน ไม่ควรใช้ค่า FDR หรือ Coverage ของโครงการใดโครงการหนึ่งเป็นตัวแทนของประสิทธิภาพทั้งหมด

4.4 ผลการทดลอง Gemini 3.8 Flash

Gemini 3.8 Flash ถูกนำมาใช้ในการสร้างชุดทดสอบระดับ Unit Test สำหรับโครงการภาษา Java ในชุดข้อมูล Defects4J โดยการทดลองครอบคลุมทั้งหมด 8 โครงการ ได้แก่ Chart, Collections, Compress, JacksonCore, JacksonDatabind, Jsoup, Lang และ Mockito ผลการประเมินพิจารณาจากความสามารถในการตรวจจับข้อบกพร่องจริงของแต่ละโครงการ และการวัด Code Coverage ของชุดทดสอบที่สร้างขึ้น โดยใช้ตัวชี้วัด Fault Detection Rate (FDR), Line Coverage และ Condition Coverage ผลการทดลองแสดงดังตารางที่ 9

ตารางที่ 9 ผลการตรวจจับข้อบกพร่องและ Code Coverage จากชุดทดสอบที่สร้างโดย Gemini 3.8 Flash

Project	จำนวน Bug	Bug ที่ ตรวจจับได้	Bug ที่ ตรวจจับไม่ได้	FDR	Line Coverage	Condition Coverage
Chart	26	21	5	80.77%	26.04%	13.25%
Collections	28	26	2	92.86%	23.83%	22.39%
Compress	47	39	5	88.64%*	31.26%	22.85%
JacksonCore	26	15	11	57.69%	19.92%	11.76%
JacksonDatabind	111	66	41	61.68%	30.00%	18.89%
Jsoup	93	78	12	86.67%*	34.91%	23.99%
Lang	61	59	2	96.72%	24.67%	17.40%
Mockito	38	28	10	73.68%	49.22%	34.16%
ค่าเฉลี่ย	—	—	—	79.84%	29.98%	20.59%

จากผลการทดลอง Gemini 3.8 Flash สามารถตรวจจับข้อบกพร่องได้ทั้งหมด 332 รายการ จาก Bug ที่สามารถนำมาวัดผลได้ 420 รายการ จากจำนวน Bug ทั้งหมด 430 รายการ โดยมี Bug ที่ไม่สามารถตรวจจับได้ 88 รายการ และมี Bug อีก 10 รายการที่ไม่สามารถนำมาประเมินผลได้ เนื่องจากข้อจำกัดในการทดสอบ ดังนั้นเมื่อคำนวณ FDR รวมจาก Bug ที่สามารถวัดผลได้ พบว่ามีค่า 79.05% เมื่อพิจารณาค่า FDR เป็นรายโครงการ พบว่า Lang มีค่า FDR สูงที่สุดที่ 96.72% โดยสามารถตรวจจับ Bug ได้ 59 จาก 61 รายการ รองลงมาคือ Collections ที่มีค่า FDR 92.86% และ Compress ที่มีค่า FDR 88.64% ขณะที่ JacksonCore มีค่า FDR ต่ำที่สุดที่ 57.69% โดยสามารถตรวจจับ Bug ได้ 15 จาก 26 รายการ

สำหรับ Compress มี Bug ที่ไม่สามารถนำมาประเมินผลได้ 3 รายการ ดังนั้นค่า FDR 88.64% คำนวณจาก Bug ที่สามารถวัดผลได้ 44 รายการ โดยตรวจจับได้ 39 รายการ และไม่สามารถตรวจจับได้ 5 รายการ ส่วน Jsoup มี Bug ที่สามารถนำมาประเมินผลได้ 3 รายการ ทำให้สามารถนำ Bug จำนวน 90 รายการมาคำนวณ FDR โดยตรวจจับได้ 78 รายการ คิดเป็น 86.67%

นอกจากการประเมินความสามารถในการตรวจจับข้อบกพร่องแล้ว ยังได้วัด Code Coverage ของชุดทดสอบที่สร้างโดย Gemini 3.8 Flash โดยพิจารณาจาก Line Coverage และ Condition Coverage ผลการทดลองพบว่า ค่าเฉลี่ย Line Coverage ของทั้ง 8 โครงการอยู่ที่ 29.98% และค่าเฉลี่ย Condition Coverage อยู่ที่ 20.59%

เมื่อพิจารณาเป็นรายโครงการ พบว่า Mockito มี Line Coverage สูงที่สุดที่ 49.22% และ Condition Coverage สูงที่สุดที่ 34.16% รองลงมาในด้าน Line Coverage คือ Jsoup ที่ 34.91% และ Compress ที่ 31.26% ส่วน JacksonCore มี Line Coverage และ Condition Coverage ต่ำที่สุดที่ 19.92% และ 11.76% ตามลำดับ

ผลการทดลองแสดงให้เห็นว่า Gemini 3.8 Flash สามารถสร้างชุดทดสอบระดับ Unit Test ที่สามารถตรวจจับข้อบกพร่องจริงใน Defects4J ได้ และสามารถครอบคลุมส่วนต่าง ๆ ของ Source Code ในระดับหนึ่ง อย่างไรก็ตาม ค่า FDR และ Code Coverage มีความแตกต่างกันในแต่ละโครงการ โดยค่า Coverage ที่สูงไม่ได้หมายความว่ามีความสามารถในการตรวจจับข้อบกพร่องสูงตามไปด้วย ตัวอย่างเช่น Mockito มี Line Coverage และ Condition Coverage สูงที่สุดในชุดการทดลองของ Gemini แต่มี FDR 73.68% ขณะที่ Lang มี FDR สูงถึง 96.72% แต่มี Line Coverage 24.67% และ Condition Coverage 17.40% แสดงให้เห็นว่า Code Coverage และความสามารถในการตรวจจับข้อบกพร่องเป็นตัวชี้วัดที่สะท้อนประสิทธิภาพของชุดทดสอบในคนละด้าน

ทั้งนี้ ค่า FDR 79.84% ในตารางเป็นค่าเฉลี่ยของ FDR รายโครงการ ขณะที่ FDR รวม 79.05% คำนวณจากจำนวน Bug ที่ตรวจจับได้เทียบกับจำนวน Bug ที่สามารถนำมาวัดผลได้ทั้งหมด จึงมีค่าแตกต่างกันเนื่องจากการคำนวณที่ต่างกัน

4.5 ผลการวัด Test Coverage

สำหรับผลการทดลอง Gemini 3.8 Flash ที่มีข้อมูลในปัจจุบัน ค่าเฉลี่ย Line Coverage อยู่ในช่วง 19.9%–49.2% และ Condition Coverage อยู่ในช่วง 11.8%–34.2%

สำหรับ Claude มีผลการวัดเพิ่มเติมดังนี้

- Cli มี Line Coverage เฉลี่ย 42.4% และ Condition Coverage เฉลี่ย 30.5%
- JacksonXml มี Line Coverage เฉลี่ย 27.1% และ Condition Coverage เฉลี่ย 19.6%
- Time มี Line Coverage เฉลี่ย 18.3% และ Condition Coverage เฉลี่ย 13.2%

Coverage ที่วัดเป็น Coverage ของคลาสที่เกี่ยวข้องกับ Bug ไม่ใช่ Coverage ของทั้งโครงการ

4.6 ผลการวัด Fault Detection

การประเมินความสามารถในการตรวจจับข้อบกพร่องพิจารณาจากเงื่อนไขที่กำหนดให้ชุดทดสอบต้องสามารถ Compile ได้ทั้ง Buggy Version และ Fixed Version โดยชุดทดสอบต้องทำให้ Buggy Version ล้มเหลว และ Fixed Version ผ่านการทดสอบ จึงจะถือว่าสามารถตรวจจับข้อบกพร่องได้

ผลการทดลองของ Gemini 3.8 Flash พบว่าสามารถตรวจจับข้อบกพร่องได้จำนวน 106 จาก 131 Bug ที่สามารถนำมาประเมินผลได้ คิดเป็น Fault Detection Rate (FDR) เท่ากับ 80.92% โดยเมื่อพิจารณาแยกตามโครงการ พบว่า Collections มีค่า FDR สูงที่สุดเท่ากับ 92.86% ขณะที่ JacksonCore มีค่า FDR ต่ำที่สุดเท่ากับ 57.69% ผลการทดลองดังกล่าวแสดงให้เห็นว่าความสามารถในการตรวจจับข้อบกพร่องของชุดทดสอบที่สร้างขึ้นมีความแตกต่างกันตามลักษณะของโครงการและข้อบกพร่อง

สำหรับ Claude AI Sonnet 5.5 ผลการประเมินค่า FDR จำแนกตามโครงการได้ดังตารางที่ 10

ตารางที่ 10 ค่า Fault Detection Rate (FDR) ของ Claude Sonnet 5.5 จำแนกตามโครงการ

โครงการ	Fault Detection Rate (FDR)
Codec	88.89%
Csv	81.25%
Gson	94.44%
JXPath	81.82%
Cli	92.31%
JacksonXml	83.33%
Time	100.00%

จากผลการทดลองพบว่า Time มีค่า FDR สูงที่สุดเท่ากับ 100.00% รองลงมาคือ Gson เท่ากับ 94.44% และ Cli เท่ากับ 92.31% ขณะที่ Csv มีค่า FDR ต่ำที่สุดเท่ากับ 81.25% เมื่อพิจารณาผลของ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ร่วมกัน พบว่าค่า Fault Detection Rate มีความแตกต่างกันในแต่ละโครงการ

อย่างไรก็ตาม การเปรียบเทียบค่า FDR ระหว่าง Claude AI Sonnet 5.5 และ Gemini 3.8 Flash จำเป็นต้องพิจารณาเงื่อนไขของการทดลองร่วมด้วย เนื่องจากทั้งสองแนวทางไม่ได้ถูกทดลองกับชุดโครงการและข้อบกพร่องเดียวกันทั้งหมด ดังนั้นค่า FDR ที่ได้จึงไม่สามารถใช้เป็นหลักฐานเพียงอย่างเดียวในการสรุปว่าเครื่องมือใดมีประสิทธิภาพในการตรวจจับข้อบกพร่องสูงกว่า การวิเคราะห์จึงควรพิจารณาพร้อมกับจำนวน Bug ที่สามารถวัดผลได้ ลักษณะของโครงการ Test Coverage และข้อจำกัดที่เกิดขึ้นระหว่างการทดลองด้วย

บทที่ 5

วิเคราะห์และอภิปรายผล

5.1 การเปรียบเทียบ Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing

Whole Test Suite Generation (WTS) และ Coverage-guided Greybox Fuzzing ใช้กลไกสร้างชุดทดสอบต่างกัน โดย WTS ใช้ Search-Based Software Testing เพื่อปรับปรุงชุดทดสอบตามค่า Fitness เช่น Coverage ส่วน Greybox Fuzzing ใช้ Coverage ที่เกิดระหว่างการทำงานของโปรแกรมเป็น Feedback สำหรับคัดเลือก Input ที่ค้นพบเส้นทางใหม่ แล้วนำ Input เหล่านั้นไปใช้เป็น Seed ในการสร้างข้อมูลทดสอบต่อไป

ผล WTS ที่รายงานในบทที่ 4 มี Line Coverage เฉลี่ย 74.50% ในรอบที่ 1 และ 64.60% ในรอบที่ 2 ส่วนเวลาเฉลี่ยอยู่ที่ 66.4 และ 41.7 วินาทีต่อคลาสตามลำดับ สำหรับ Coverage-guided Greybox Fuzzing จากไฟล์ผลการรันของ Algorithm 2 พบ Line Coverage เฉลี่ย 23.66% ในรอบที่ 1 และ 23.62% ในรอบที่ 2 เวลาเฉลี่ยอยู่ที่ 58.11 และ 54.45 วินาทีต่อรายการรันตามลำดับ

ตาราง 11 การเปรียบเทียบ Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing

วิธีการ	รอบที่ 1: Line Coverage เฉลี่ย	รอบที่ 1: เวลาเฉลี่ย	รอบที่ 2: Line Coverage เฉลี่ย	รอบที่ 2: เวลาเฉลี่ย
Whole Test Suite Generation	74.50%	66.4 วินาที/ คลาส	64.60%	41.7 วินาที/ คลาส
Coverage-guided Greybox Fuzzing (Algorithm 2)	23.66%	58.11 วินาที/ รายการรัน	23.62%	54.45 วินาที/ รายการรัน

จากข้อมูลที่มี WTS มีค่า Line Coverage เฉลี่ยสูงกว่า Greybox Fuzzing ทั้งสองรอบ ขณะที่เวลาเฉลี่ยที่รายงานของ Greybox Fuzzing ต่ำกว่าในทั้งสองรอบ อย่างไรก็ตาม ยังไม่ควรสรุปว่า WTS มีประสิทธิภาพด้าน Coverage สูงกว่า หรือ Greybox Fuzzing ใช้เวลาน้อยกว่าโดยตรง เพราะหน่วยและวิธีคำนวณต่างกัน: เวลา WTS วัดต่อคลาส ส่วนเวลา Greybox Fuzzing วัดต่อรายการ bug ที่รัน และ Line Coverage ของ Greybox Fuzzing เฉลี่ยจากรายการที่มีรายงาน coverage โดยรวมรายการสถานะ CRASH ที่ยังมีข้อมูล coverage ด้วย

นอกจากนี้ ขอบเขตของข้อมูลยังต่างกัน โดยไฟล์ Greybox Fuzzing มี 856 รายการในแต่ละรอบ แต่มีเพียง 660 รายการในรอบที่ 1 และ 659 รายการในรอบที่ 2 ที่มีข้อมูล Line Coverage เพียงพอสำหรับคำนวณ

ส่วนรายการที่ไม่มีรายงานหรือไม่มีข้อมูล target class จะไม่ถูกรวมเป็น 0% การเปรียบเทียบเชิงตัวเลขจึงควรถือเป็นแนวโน้มจากผลที่มี มากกว่าการเปรียบเทียบแบบเงื่อนไขเดียวกัน

โดยสรุป WTS มีค่า Coverage ที่รายงานสูงกว่าในชุดผลการทดลองนี้ ส่วน Greybox Fuzzing แสดงค่า Coverage ใกล้เคียงกันระหว่างสองรอบ แต่ข้อมูลที่มีอยู่ยังไม่เพียงพอสำหรับเปรียบเทียบประสิทธิภาพโดยตรงอย่างยุติธรรม ควรใช้หน่วยวัดและชุดรายการเดียวกัน รวมทั้งกำหนดวิธีนับสถานะการรันและ Coverage ให้ตรงกันก่อนสรุปว่าแนวทางใดเหมาะสมกว่าครับ

5.2 การเปรียบเทียบ Claude Sonnet 5.5 และ Gemini 3.8 Flash

ผลการทดลองของ Claude Sonnet 5.5 และ Gemini 3.8 Flash แสดงให้เห็นว่า Generative AI สามารถนำมาใช้ในการสร้างชุดทดสอบระดับ Unit Test สำหรับโครงการภาษา Java ใน Defects4J และสามารถสร้างชุดทดสอบที่ตรวจจับข้อบกพร่องจริงได้ อย่างไรก็ตาม ผลลัพธ์ของทั้งสองเครื่องมือมีความแตกต่างกันทั้งในด้าน Fault Detection Rate (FDR) และ Code Coverage ดังนั้นการเปรียบเทียบจึงต้องพิจารณาพร้อมกับเงื่อนไขของชุดข้อมูลที่ใช้ในการทดลอง

ในการทดลองครั้งนี้ Claude Sonnet 5.5 ถูกนำไปทดสอบกับ 9 โครงการ ได้แก่ Cli, Closure, Codec, Csv, Gson, JacksonXml, JXPath, Math และ Time มี Bug ทั้งหมด 424 รายการ โดยสามารถตรวจจับ Bug ได้ 306 รายการ และมี Bug ที่ไม่สามารถตรวจจับได้ 100 รายการ ส่วน Bug อีก 18 รายการไม่สามารถนำมาประเมินผลได้ ส่งผลให้ FDR รวมจาก Bug ที่สามารถวัดผลได้อยู่ที่ 75.37% และมีค่าเฉลี่ย FDR รายโครงการเท่ากับ 85.50% โดยมีค่าเฉลี่ย Line Coverage 36.26% และ Condition Coverage 26.45%

สำหรับ Gemini 3.8 Flash ถูกนำไปทดสอบกับ 8 โครงการ ได้แก่ Chart, Collections, Compress, JacksonCore, JacksonDatabind, Jsoup, Lang และ Mockito มี Bug ทั้งหมด 430 รายการ โดยสามารถตรวจจับ Bug ได้ 332 รายการ และมี Bug ที่ไม่สามารถตรวจจับได้ 88 รายการ ส่วน Bug อีก 10 รายการไม่สามารถนำมาประเมินผลได้ ส่งผลให้ FDR รวมจาก Bug ที่สามารถวัดผลได้อยู่ที่ 79.05% และมีค่าเฉลี่ย FDR รายโครงการเท่ากับ 79.84% ขณะที่ค่าเฉลี่ย Line Coverage อยู่ที่ 29.98% และ Condition Coverage อยู่ที่ 20.59%

เมื่อพิจารณาจากค่าเฉลี่ย FDR รายโครงการ Claude Sonnet 5.5 มีค่าเฉลี่ย 85.50% ซึ่งสูงกว่า Gemini 3.8 Flash ที่มีค่าเฉลี่ย 79.84% ขณะที่ Claude Sonnet 5.5 มีค่าเฉลี่ย Line Coverage และ Condition Coverage เท่ากับ 36.26% และ 26.45% ตามลำดับ ซึ่งสูงกว่า Gemini 3.8 Flash ที่มีค่าเฉลี่ย 29.98% และ 20.59% ตามลำดับ อย่างไรก็ตาม ค่า FDR รวมที่คำนวณจากจำนวน Bug ที่สามารถวัดผลได้กลับพบว่า Gemini มีค่า 79.05% สูงกว่า Claude ที่มีค่า 75.37%

ความแตกต่างระหว่างค่าเฉลี่ย FDR รายโครงการและ FDR รวมเกิดจากวิธีการคำนวณที่แตกต่างกัน โดยค่าเฉลี่ย FDR รายโครงการให้ความสำคัญกับแต่ละ Project ในสัดส่วนเท่ากัน ขณะที่ FDR รวมพิจารณาจาก

จำนวน Bug ที่สามารถวัดผลได้ทั้งหมด จึงได้รับอิทธิพลจาก Project ที่มีจำนวน Bug มาก ตัวอย่างเช่น Closure ของ Claude มี Bug จำนวนมากและมี FDR ค่อนข้างต่ำ ส่งผลต่อ FDR รวมของ Claude มากกว่าการคำนวณค่าเฉลี่ยของแต่ละ Project

นอกจากนี้ การเปรียบเทียบโดยตรงระหว่าง Claude Sonnet 5.5 และ Gemini 3.8 Flash มีข้อจำกัด เนื่องจากทั้งสองเครื่องมือไม่ได้ถูกทดสอบกับ Project ชุดเดียวกัน โดย Claude ครอบคลุม 9 โครงการ ส่วน Gemini ครอบคลุม 8 โครงการ และทั้งสองชุดมี Project ที่แตกต่างกัน ดังนั้นค่าเฉลี่ย FDR และ Coverage ที่แตกต่างกันจึงไม่สามารถใช้เป็นหลักฐานเพียงพอในการสรุปว่าเครื่องมือใดมีประสิทธิภาพเหนือกว่าอีกเครื่องมือหนึ่ง เนื่องจากลักษณะของ Source Code จำนวน Bug และความซับซ้อนของ Bug ในแต่ละ Project อาจส่งผลต่อผลการทดลอง

เมื่อพิจารณาด้าน Code Coverage พบว่า Claude มีค่าเฉลี่ย Line Coverage และ Condition Coverage สูงกว่า Gemini ในชุด Project ที่นำมาทดลอง แต่ผลดังกล่าวไม่ได้หมายความว่าชุดทดสอบของ Claude สามารถตรวจจับ Bug ได้มากกว่าเสมอไป เนื่องจาก Coverage เป็นตัวชี้วัดการเข้าถึงส่วนต่าง ๆ ของ Source Code ขณะที่ FDR เป็นตัวชี้วัดความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่อง ทั้งสองตัวชี้วัดจึงสะท้อนประสิทธิภาพของชุดทดสอบในคนละด้าน

จากผลการทดลองจึงสามารถกล่าวได้ว่า Claude Sonnet 5.5 มีค่าเฉลี่ย FDR และ Code Coverage รายโครงการสูงกว่า Gemini 3.8 Flash ในชุดการทดลองของตนเอง ขณะที่ Gemini 3.8 Flash มี FDR รวมแบบพิจารณาจากจำนวน Bug ที่สามารถวัดผลได้สูงกว่า Claude อย่างไรก็ตาม ผลดังกล่าวควรใช้เพื่อแสดงแนวโน้มของผลการทดลองมากกว่าการจัดอันดับประสิทธิภาพของเครื่องมือ เนื่องจากทั้งสองเครื่องมือไม่ได้ถูกทดสอบภายใต้ชุด Project และ Bug เดียวกัน

ดังนั้น หากต้องการเปรียบเทียบประสิทธิภาพของ Claude Sonnet 5.5 และ Gemini 3.8 Flash โดยตรง ควรให้ทั้งสองเครื่องมือสร้างชุดทดสอบจาก Project และ Bug ชุดเดียวกัน โดยควบคุม Prompt, Configuration, จำนวนครั้งในการทดลอง และเงื่อนไขการประเมินให้เหมือนกันหรือใกล้เคียงกันมากที่สุด เพื่อให้ความแตกต่างของผลลัพธ์สะท้อนความแตกต่างระหว่างเครื่องมือมากกว่าความแตกต่างของชุดข้อมูลที่ใช้ในการทดลอง

5.3 การเปรียบเทียบทั้ง 4 แนวทาง

การทดลองนี้ประกอบด้วย 4 แนวทาง ได้แก่ Whole Test Suite Generation (WTS), Coverage-guided Greybox Fuzzing, Claude Sonnet 5.5 และ Gemini 3.8 Flash โดยแต่ละแนวทางมีหลักการสร้างชุดทดสอบและลักษณะการทำงานที่แตกต่างกัน ดังแสดงในตารางที่ 12

ตารางที่ 12 การเปรียบเทียบลักษณะของแนวทางการสร้างชุดทดสอบทั้ง 4 แนวทาง

แนวทาง	หลักการ	จุดเด่น	ข้อจำกัด
WTS / EvoSuite	Search-Based Software Testing	ค้นหาและปรับปรุง Test Suite โดยใช้ Fitness และ Coverage เป็นส่วนหนึ่งของกระบวนการ Search	ผลลัพธ์ขึ้นอยู่กับ Search Budget และสภาพแวดล้อมในการทดลอง
Coverage-guided Greybox Fuzzing	Coverage Feedback และ Mutation	สามารถสร้าง Input จำนวนมากและใช้ Coverage เป็น Feedback เพื่อสำรวจเส้นทางการทำงานใหม่	ต้องออกแบบ Harness และกำหนด Oracle ที่เหมาะสม รวมถึงผล Coverage ที่ได้ขึ้นอยู่กับลักษณะของ Target Program
Claude Sonnet 5.5	AI-Assisting Tool ที่ใช้ Generative AI	สามารถวิเคราะห์ Source Code และข้อมูลของ Bug เพื่อสร้าง Test Code จาก Prompt	Test ที่สร้างขึ้นอาจ Compile ไม่ผ่าน มี Assertion ที่ไม่เหมาะสม หรือไม่สามารถนำไปประเมินผลได้ในบางกรณี
Gemini 3.8 Flash	AI-Assisting Tool ที่ใช้ Generative AI	สามารถวิเคราะห์ Source Code และข้อมูลของ Bug เพื่อสร้าง Test Code	มีข้อจำกัดด้าน Token และ API และอาจเกิด Response ที่ไม่สมบูรณ์หรือไม่สามารถสร้าง Test ที่ใช้งานได้

เมื่อพิจารณาหลักการทำงานของทั้ง 4 แนวทาง พบว่า WTS และ Coverage-guided Greybox Fuzzing เป็นแนวทางที่ใช้ Feedback จากการทำงานของโปรแกรม เป็นส่วนสำคัญของกระบวนการสร้างและปรับปรุงข้อมูลทดสอบ โดย WTS ใช้ Search-Based Software Testing เพื่อค้นหาและปรับปรุง Test Suite ตาม Fitness ที่กำหนด ขณะที่ Coverage-guided Greybox Fuzzing ใช้ Coverage ที่เกิดจากการ Execute เป็น Feedback เพื่อคัดเลือก Input ที่สามารถค้นพบเส้นทางการทำงานใหม่และนำไปใช้เป็น Seed สำหรับการสร้าง Input ต่อไป

จากผลการทดลองในหัวข้อ 5.1 พบว่า WTS มีค่า Line Coverage เฉลี่ยที่รายงานสูงกว่า Coverage-guided Greybox Fuzzing ทั้งในรอบที่ 1 และรอบที่ 2 อย่างไรก็ตาม ผลดังกล่าวยังไม่สามารถใช้สรุปว่า WTS มีประสิทธิภาพเหนือกว่า Greybox Fuzzing โดยตรง เนื่องจากวิธีการวัดและหน่วยของเวลาที่ใช้แตกต่างกัน โดย WTS รายงานเวลาเป็นต่อคลาส ขณะที่ Greybox Fuzzing รายงานเวลาเป็นต่อรายการรัน นอกจากนี้ Greybox Fuzzing ยังมีรายการที่ไม่มีข้อมูล Coverage เพียงพอสำหรับการคำนวณ และรายการดังกล่าวไม่ได้ถูกนับเป็นค่า 0% ดังนั้นผลการเปรียบเทียบในส่วนนี้จึงควรพิจารณาเป็นแนวโน้มจากข้อมูลที่สามารถนำมาวิเคราะห์ได้

สำหรับ Claude Sonnet 5.5 และ Gemini 3.8 Flash เป็นแนวทางที่แตกต่างจากสองวิธีแรก เนื่องจากใช้ Generative AI ในการวิเคราะห์ Source Code และข้อมูลบริบทของ Bug เพื่อสร้าง Test Code โดยไม่จำเป็นต้องอาศัยกระบวนการ Search หรือ Mutation ในลักษณะเดียวกับ WTS และ Greybox Fuzzing ผลการทดลองในหัวข้อ 5.2 พบว่า Claude Sonnet 5.5 มีค่าเฉลี่ย FDR รายโครงการเท่ากับ 85.50% สูงกว่า Gemini 3.8 Flash ที่มีค่าเฉลี่ย 79.84% และมีค่าเฉลี่ย Line Coverage และ Condition Coverage สูงกว่า Gemini เช่นเดียวกัน อย่างไรก็ตาม เมื่อคำนวณ FDR จากจำนวน Bug ที่สามารถวัดผลได้ทั้งหมด Gemini มีค่า FDR รวม 79.05% สูงกว่า Claude ที่มีค่า 75.37%

ความแตกต่างดังกล่าวแสดงให้เห็นว่า วิธีการคำนวณตัวชี้วัดมีผลต่อผลการเปรียบเทียบ โดยค่าเฉลี่ย FDR รายโครงการให้น้ำหนักแต่ละ Project เท่ากัน ขณะที่ FDR รวมให้น้ำหนักตามจำนวน Bug ที่สามารถประเมินผลได้ นอกจากนี้ Claude Sonnet 5.5 และ Gemini 3.8 Flash ยังถูกทดลองกับชุด Project ที่แตกต่างกัน จึงไม่สามารถใช้ค่า FDR หรือ Coverage ที่ได้มาเป็นหลักฐานเพียงอย่างเดียวในการจัดอันดับว่าเครื่องมือใดมีประสิทธิภาพเหนือกว่าอีกเครื่องมือหนึ่ง

เมื่อพิจารณาภาพรวมของทั้ง 4 แนวทาง จึงพบว่าแต่ละแนวทางมีลักษณะการสร้าง Test และข้อจำกัดที่แตกต่างกัน WTS มีแนวโน้มสร้าง Test Suite ที่มี Coverage สูงจากกระบวนการ Search และการปรับปรุง Fitness ขณะที่ Coverage-guided Greybox Fuzzing มีจุดเด่นด้านการสำรวจ Input Space ผ่าน Coverage Feedback และ Mutation ส่วน Claude Sonnet 5.5 และ Gemini 3.8 Flash สามารถใช้ข้อมูลเชิงบริบทของ Source Code และ Bug เพื่อสร้าง Test Code โดยตรง ซึ่งช่วยให้กระบวนการสร้าง Test มีลักษณะแตกต่างจากวิธีที่อาศัยการค้นหาแบบอัตโนมัติหรือการ Mutation

อย่างไรก็ตาม ผลการทดลองยังแสดงให้เห็นว่าประสิทธิภาพของแต่ละแนวทางไม่ได้ขึ้นอยู่กับตัวชี้วัดเพียงด้านเดียว โดย Coverage สะท้อนระดับการเข้าถึงส่วนต่าง ๆ ของ Source Code ขณะที่ FDR สะท้อนความสามารถของ Test Suite ในการตรวจจับข้อบกพร่อง ดังนั้น Test Suite ที่มี Coverage สูงจึงไม่ได้

หมายความว่า จะสามารถตรวจจับ Bug ได้มากกว่าเสมอไป นอกจากนี้ ความสามารถในการ Compile และ Execute ของ Test รวมถึงรายการที่ไม่สามารถประเมินผลได้ ยังเป็นปัจจัยสำคัญต่อผลลัพธ์ของการทดลอง

ดังนั้น การเปรียบเทียบทั้ง 4 แนวทางในการทดลองนี้ควรใช้เพื่อแสดง ลักษณะและแนวโน้มของผลการทดลอง มากกว่าการจัดอันดับประสิทธิภาพโดยตรง เนื่องจากแต่ละแนวทางมีวิธีการวัดผล ชุดข้อมูล ขอบเขตการทดลอง และข้อจำกัดที่แตกต่างกัน โดยเฉพาะ Coverage-guided Greybox Fuzzing ที่มีหน่วยการรายงานผลแตกต่างจาก WTS และ Claude Sonnet 5.5 กับ Gemini 3.8 Flash ที่ใช้ชุด Project และ Bug แตกต่างกัน

หากต้องการเปรียบเทียบประสิทธิภาพของทั้ง 4 แนวทางโดยตรงในอนาคต ควรควบคุมให้ทุกแนวทางทำงานกับ Project และ Bug ชุดเดียวกัน กำหนดเวลาและทรัพยากรที่ใช้ให้เทียบเคียงกัน ใช้เกณฑ์เดียวกันในการนับ Test ที่ Compile และ Execute สำเร็จ รวมถึงกำหนดวิธีการคำนวณ Coverage และ FDR ให้เป็นรูปแบบเดียวกัน เพื่อให้ความแตกต่างของผลลัพธ์สะท้อนความแตกต่างของแนวทางการสร้าง Test มากกว่าความแตกต่างของเงื่อนไขการทดลอง

5.4 ความสัมพันธ์ระหว่าง Test Coverage และ Fault Detection

ผลการทดลองแสดงให้เห็นว่า Test Coverage และความสามารถในการตรวจจับข้อบกพร่องเป็นตัวชี้วัดที่สะท้อนคุณภาพของชุดทดสอบในคนละด้าน โดย Test Coverage ใช้แสดงระดับการครอบคลุมส่วนต่าง ๆ ของ Source Code ขณะที่ Fault Detection Rate (FDR) ใช้แสดงความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องที่มีอยู่ในโครงการ ดังนั้น ค่า Coverage ที่สูงจึงไม่ได้หมายความว่าชุดทดสอบจะสามารถตรวจจับข้อบกพร่องได้สูงเสมอไป

จากผลการทดลองของ Claude Sonnet 5.5 พบว่ามีค่าเฉลี่ย Line Coverage 36.26% และ Condition Coverage 26.45% ขณะที่ค่าเฉลี่ย FDR รายโครงการอยู่ที่ 85.50% ส่วน Gemini 3.8 Flash มีค่าเฉลี่ย Line Coverage 29.98% และ Condition Coverage 20.59% และมีค่าเฉลี่ย FDR รายโครงการ 79.84% ผลดังกล่าวแสดงให้เห็นว่า Coverage และ FDR ไม่ได้เปลี่ยนแปลงไปในทิศทางเดียวกันในทุกโครงการ

ตัวอย่างที่เห็นได้ชัดคือผลการทดลองของ Gemini 3.8 Flash โดย Mockito มี Line Coverage สูงที่สุดที่ 49.22% และ Condition Coverage สูงที่สุดที่ 34.16% แต่มี FDR เพียง 73.68% ในขณะที่ Lang มี Line Coverage 24.67% และ Condition Coverage 17.40% ซึ่งต่ำกว่า Mockito แต่มี FDR สูงถึง 96.72% แสดงให้เห็นว่าการที่ชุดทดสอบสามารถเข้าถึง Source Code ได้มาก ไม่ได้หมายความว่า จะสามารถตรวจจับข้อบกพร่องได้มากตามไปด้วย

ในทำนองเดียวกัน ผลการทดลองของ Claude Sonnet 5.5 พบว่า XPath มี Line Coverage สูงที่สุดที่ 44.80% และ Condition Coverage 33.71% แต่มี FDR 81.82% ขณะที่ Time มี Line Coverage เพียง 20.14% และ Condition Coverage 14.94% แต่มี FDR 100.00% จากผลดังกล่าวแสดงให้เห็นว่า Test Coverage และ Fault Detection สามารถให้ผลแตกต่างกันได้ แม้จะเป็นชุดทดสอบที่สร้างโดยเครื่องมือเดียวกัน

สาเหตุหนึ่งที่ Coverage และ Fault Detection ให้ผลแตกต่างกัน คือ Coverage พิจารณาว่าชุดทดสอบสามารถเข้าถึงส่วนของ Source Code ได้บ้าง แต่ไม่ได้พิจารณาโดยตรงว่าผลลัพธ์ที่เกิดขึ้นจากการทดสอบสามารถตรวจจับพฤติกรรมที่ผิดพลาดของโปรแกรมได้หรือไม่ ชุดทดสอบจึงอาจมี Coverage สูงจากการ Execute Code หรือ Condition จำนวนมาก แต่หากไม่มี Input หรือ Assertion ที่เหมาะสมกับลักษณะของ Bug ชุดทดสอบนั้นก็อาจไม่สามารถตรวจจับข้อบกพร่องได้

ในทางกลับกัน ชุดทดสอบที่มี Coverage ไม่สูงมากอาจสามารถตรวจจับข้อบกพร่องได้ หากมี Input และ Assertion ที่สามารถทำให้เกิดผลลัพธ์แตกต่างกันระหว่าง Buggy Version และ Fixed Version ดังนั้น การมี Coverage ในระดับสูงจึงไม่ควรถูกใช้เป็นหลักฐานเพียงอย่างเดียวในการสรุปว่าชุดทดสอบมีประสิทธิภาพในการตรวจจับข้อบกพร่องสูง

จากผลการทดลองครั้งนี้จึงควรพิจารณา Test Coverage และ Fault Detection Rate ร่วมกัน โดย Line Coverage และ Condition Coverage ใช้สำหรับประเมินความครอบคลุมของ Source Code ส่วน FDR ใช้ประเมินความสามารถในการตรวจจับข้อบกพร่อง การใช้ตัวชี้วัดทั้งสองกลุ่มร่วมกันช่วยให้การประเมินคุณภาพของชุดทดสอบมีความครอบคลุมมากกว่าการพิจารณา Coverage เพียงอย่างเดียว

อย่างไรก็ตาม ผลการเปรียบเทียบระหว่าง Claude Sonnet 5.5 และ Gemini 3.8 Flash ในการทดลองนี้ยังไม่สามารถใช้สรุปความสัมพันธ์เชิงสถิติระหว่าง Coverage และ FDR ได้โดยตรง เนื่องจากทั้งสองเครื่องมือถูกทดสอบกับชุด Project ที่แตกต่างกัน และการทดลองไม่ได้ออกแบบมาเพื่อศึกษาความสัมพันธ์เชิงสถิติโดยเฉพาะ ดังนั้น ผลที่นำเสนอในส่วนนี้จึงเป็นการวิเคราะห์แนวโน้มจากผลการทดลองที่ได้ มากกว่าการสรุปว่าค่า Coverage และ FDR มีความสัมพันธ์กันในเชิงสถิติอย่างแน่นอน.

5.5 การวิเคราะห์รูปแบบความล้มเหลว

จากผลการทดลองพบว่า ผลลัพธ์จากการสร้างชุดทดสอบด้วย AI-Assisting Tools ไม่ได้มีเพียงกรณีที่ สามารถตรวจจับข้อบกพร่องและกรณีที่ไม่สามารถตรวจจับข้อบกพร่องเท่านั้น แต่ยังพบกรณีที่ไม่สามารถนำผลการทดลองมาประเมินได้ เนื่องจากปัญหาของ Test Code สภาพแวดล้อมในการทดสอบ หรือความสมบูรณ์ของ Response ที่ได้รับจาก AI-Assisting Tools การจำแนกรูปแบบความล้มเหลวจึงมีความสำคัญต่อการวิเคราะห์ผลการทดลองและการคำนวณ Fault Detection Rate เนื่องจากช่วยแยกความล้มเหลวที่เกิดจากความสามารถของชุดทดสอบออกจากปัญหาที่ไม่สามารถใช้ประเมินความสามารถในการตรวจจับข้อบกพร่องได้โดยตรง

5.5.1 Test ผ่านทั้ง Buggy Version และ Fixed Version กรณีที่ชุดทดสอบสามารถ Compile และ Execute ได้ทั้ง Buggy Version และ Fixed Version แต่ผลการทดสอบไม่สามารถแสดงความแตกต่างระหว่างทั้งสอง Version ได้ จะถือว่าชุดทดสอบไม่สามารถตรวจจับข้อบกพร่องเป้าหมาย และจัดอยู่ในกลุ่ม Not Detected

กรณีดังกล่าวแสดงให้เห็นว่า การที่ Test สามารถทำงานได้สำเร็จไม่ได้หมายความว่า Test จะสามารถตรวจจับข้อบกพร่องได้ เนื่องจากชุดทดสอบอาจไม่ได้กำหนด Input หรือ Assertion ที่สามารถแสดงพฤติกรรมที่แตกต่างกันระหว่าง Buggy Version และ Fixed Version ดังนั้น Test ที่สามารถ Execute ได้จึงยังจำเป็นต้องพิจารณาผลลัพธ์ของการทดสอบร่วมด้วย

5.5.2 Test ล้มเหลวทั้ง Buggy Version และ Fixed Version กรณีที่ชุดทดสอบล้มเหลวทั้ง Buggy Version และ Fixed Version ไม่สามารถใช้ผลการทดสอบเพื่อยืนยันได้ว่าความล้มเหลวนั้นเกิดจากข้อบกพร่องเป้าหมาย เนื่องจากสาเหตุอาจมาจากชุดทดสอบเอง หรือปัจจัยอื่นที่เกี่ยวข้องกับสภาพแวดล้อมในการทดสอบ

จากการทดลองพบว่าความล้มเหลวในลักษณะดังกล่าวอาจเกี่ยวข้องกับ API, Helper, Resource, Classpath หรือ Environment ของโครงการ ดังนั้นจึงจำเป็นต้องพิจารณาสาเหตุของความล้มเหลวก่อนจัดประเภทผลการทดลอง เพื่อป้องกันไม่ให้ความล้มเหลวที่ไม่ได้เกิดจาก Bug เป้าหมายถูกนำไปนับเป็น Fault Detection

กรณีที่ไม่สามารถระบุผลการตรวจจับข้อบกพร่องได้จากข้อจำกัดดังกล่าว จะถูกแยกออกจาก Bug ที่สามารถนำมาคำนวณ FDR เพื่อให้ค่าที่ได้สะท้อนเฉพาะผลการทดสอบที่สามารถประเมินได้

5.5.3 Compile ไม่ผ่าน จากการทดลองพบกรณีที่ Test Code ที่สร้างโดย AI-Assisting Tools ไม่สามารถ Compile ได้ โดยสาเหตุส่วนหนึ่งเกิดจากการเรียกใช้ API หรือ Method ที่ไม่มีอยู่จริง การใช้งาน API ไม่ถูกต้อง หรือการอ้างอิง Class ที่ไม่สามารถใช้งานได้ ใน Environment ของโครงการ กรณีที่ Compile ไม่ผ่านเนื่องจากข้อผิดพลาดของ Test Code จะถือว่าชุดทดสอบไม่สามารถนำไปใช้ตรวจสอบ Bug ได้ และจัดอยู่ในกลุ่ม Not Detected เนื่องจากไม่มีผลการ Execute Test ที่สามารถใช้ประเมินการตรวจจับข้อบกพร่องได้

ในทางกลับกัน หากสาเหตุที่ทำให้ Compile หรือ Execute ไม่ได้เกิดจาก Helper, Classpath, Resource หรือ Environment ของโครงการ และไม่สามารถแก้ไขหรือดำเนินการทดสอบต่อได้ จะถูกแยกออกเป็นกรณีไม่สามารถวัดผลได้ เพื่อไม่ให้ปัญหาจากสภาพแวดล้อมถูกนำไปตีความว่าเป็นความล้มเหลวของความสามารถในการสร้าง Test ของ AI โดยตรง

5.5.4 Response ไม่สมบูรณ์ ในการทดลอง Gemini 3.8 Flash มีการกำหนดค่า max_tokens เท่ากับ 4096 Token โดยกระบวนการสร้าง Response ของโมเดลอยู่ภายใต้ข้อจำกัดดังกล่าว จากการทดลองพบว่าในบางกรณีโมเดลใช้ Token จนถึงขีดจำกัดก่อนที่จะสร้าง Test Code ได้อย่างสมบูรณ์ ส่งผลให้

Response สิ้นสุดด้วย stop_reason = max_tokens และไม่สามารถนำ Test Code ที่ได้ไปใช้ในการ Compile และ Execute ได้อย่างสมบูรณ์

สำหรับโครงการ Compress พบกรณี Response ไม่สมบูรณ์จำนวน 2 Bug ได้แก่ Bug 20 และ Bug 23 โดย Response ของทั้งสองกรณีสิ้นสุดด้วย max_tokens และมีจำนวน output_tokens ใกล้เคียงกับขีดจำกัด 4096 Token แม้ว่าจะมีการเพิ่ม EFFICIENCY NOTE ใน Prompt เพื่อช่วยควบคุมการใช้ Token แล้วก็ตาม

กรณี Response ไม่สมบูรณ์ถูกแยกออกจากผลการตรวจจับข้อบกพร่อง เนื่องจากไม่มี Test Code ที่สมบูรณ์เพียงพอสำหรับนำไป Compile และ Execute นอกจากนี้ ในการทดลองครั้งนี้ไม่ได้เรียก API เข้าเพื่อสร้าง Test ใหม่ และไม่นำ Code ที่ถูกตัดระหว่างการสร้างไปใช้ในการประเมินผล ดังนั้นกรณีดังกล่าวจึงถูกจัดเป็น “ตอบไม่จบ” และไม่ถูกนำมารวมในตัวหารของ Fault Detection Rate เพื่อป้องกันไม่ให้ผลการประเมินได้รับอิทธิพลจาก Response ที่ไม่สมบูรณ์ของโมเดล

โดยสรุป รูปแบบความล้มเหลวที่พบในการทดลองสามารถแบ่งตามลักษณะของผลการทดสอบได้เป็นกรณีที่ Test ทำงานได้แต่ไม่สามารถตรวจจับ Bug, กรณีที่ Test ล้มเหลวจากปัญหาของ Test Code, กรณีที่ไม่สามารถประเมินผลได้จากข้อจำกัดของ Environment และกรณีที่ AI สร้าง Response ไม่สมบูรณ์ การแยกประเภทดังกล่าวช่วยให้การคำนวณ FDR สามารถแยกความสามารถในการตรวจจับข้อบกพร่องออกจากปัญหาที่เกิดขึ้นระหว่างกระบวนการสร้างและการทดสอบชุดทดสอบได้ชัดเจนยิ่งขึ้น.

5.6 อภิปรายผลการทดลอง

จากผลการทดลองพบว่า Automated Test Case Generation Algorithms และ AI-Assisting Tools สามารถนำมาใช้สร้างชุดทดสอบระดับ Unit Testing สำหรับโครงการภาษา Java ใน Defects4J ได้ โดยแต่ละแนวทางสามารถสร้าง Test ที่นำไปใช้ประเมินความสามารถในการตรวจจับข้อบกพร่องและวัด Code Coverage ได้ อย่างไรก็ตาม ผลลัพธ์ที่ได้มีความแตกต่างกันตามหลักการทำงานของแต่ละแนวทาง รวมถึงเงื่อนไขและขอบเขตของข้อมูลที่ใช้ในการทดลอง

เมื่อพิจารณา Whole Test Suite Generation และ Coverage-guided Greybox Fuzzing พบว่า WTS มีค่า Line Coverage เฉลี่ยที่รายงานสูงกว่า Greybox Fuzzing ทั้งในรอบที่ 1 และรอบที่ 2 ขณะที่ Greybox Fuzzing มีค่า Coverage ใกล้เคียงกันระหว่างสองรอบ อย่างไรก็ตาม ไม่สามารถสรุปได้โดยตรงว่า WTS มีประสิทธิภาพเหนือกว่า Greybox Fuzzing เนื่องจากหน่วยการวัดเวลาและวิธีการคำนวณ Coverage แตกต่างกัน รวมถึง Greybox Fuzzing มีบางรายการที่ไม่มีข้อมูล Coverage เพียงพอสำหรับการคำนวณ ดังนั้นผลการเปรียบเทียบในส่วนนี้จึงควรใช้เพื่อแสดงแนวโน้มของผลการทดลอง

สำหรับ AI-Assisting Tools ผลการทดลองพบว่า Claude Sonnet 5.5 และ Gemini 3.8 Flash สามารถวิเคราะห์ Source Code และข้อมูลบริบทของ Bug เพื่อสร้าง Test Code ที่สามารถนำไปตรวจจับข้อบกพร่องจริงได้ โดย Claude Sonnet 5.5 มีค่าเฉลี่ย FDR รายโครงการ 85.50% ขณะที่ Gemini 3.8 Flash มีค่าเฉลี่ย FDR รายโครงการ 79.84% และมีค่าเฉลี่ย Line Coverage และ Condition Coverage ต่ำกว่า Claude อย่างไรก็ตาม เมื่อพิจารณา FDR รวมจากจำนวน Bug ที่สามารถวัดผลได้ Gemini มีค่า 79.05% สูงกว่า Claude ที่มีค่า 75.37%

ความแตกต่างระหว่างค่าเฉลี่ย FDR รายโครงการและ FDR รวมแสดงให้เห็นว่า วิธีการคำนวณและจำนวน Bug ในแต่ละ Project มีผลต่อผลลัพธ์ที่ได้ โดยค่าเฉลี่ย FDR รายโครงการให้น้ำหนักแต่ละ Project เท่ากัน ขณะที่ FDR รวมได้รับอิทธิพลจาก Project ที่มีจำนวน Bug มาก นอกจากนี้ Claude Sonnet 5.5 และ Gemini 3.8 Flash ยังถูกทดลองกับ Project ที่แตกต่างกัน จึงไม่สามารถใช้ค่า FDR หรือ Coverage ดังกล่าวเพื่อจัดอันดับประสิทธิภาพของทั้งสองเครื่องมือโดยตรงได้

ในส่วน of AI-Assisting Tools ยังพบข้อจำกัดจากกระบวนการสร้าง Test เช่น Test Code ที่ Compile ไม่ผ่าน Response ที่ไม่สมบูรณ์ และ Test ที่ไม่สามารถนำมาประเมินผลได้ รวมถึงข้อจำกัดด้าน API และ Token ในบางกรณี ขณะที่ WTS และ Coverage-guided Greybox Fuzzing มีข้อจำกัดในลักษณะที่แตกต่างกัน เช่น Search Budget การออกแบบ Harness และการกำหนดกลไกสำหรับตรวจสอบผลลัพธ์ ดังนั้นความแตกต่างของผลลัพธ์จึงไม่ได้เกิดจากวิธีการสร้าง Test เพียงอย่างเดียว แต่เกี่ยวข้องกับเงื่อนไขของการทดลองและข้อจำกัดของแต่ละแนวทางด้วย

เมื่อพิจารณาผลการทดลองโดยรวมจึงพบว่าแต่ละแนวทางมีจุดเด่นแตกต่างกัน WTS มีจุดเด่นด้านการค้นหาและปรับปรุง Test Suite โดยใช้ Search และ Coverage Feedback ขณะที่ Coverage-guided Greybox Fuzzing มีจุดเด่นด้านการสำรวจ Input Space และค้นหาเส้นทางการทำงานใหม่ ส่วน Claude Sonnet 5.5 และ Gemini 3.8 Flash มีความสามารถในการวิเคราะห์ Source Code และข้อมูลของ Bug เพื่อสร้าง Test Code โดยตรง ดังนั้นจึงไม่ควรพิจารณาประสิทธิภาพของทั้ง 4 แนวทางจาก Coverage หรือ FDR เพียงตัวเดียว

จากข้อสังเกตดังกล่าว สามารถนำไปสู่แนวทางการศึกษาการทำงานร่วมกันระหว่างวิธีการต่าง ๆ เช่น การใช้ AI-Assisting Tools เพื่อสร้าง Test เบื้องต้น แล้วใช้ Search-Based Software Testing หรือ Fuzzing เพื่อเพิ่มและปรับปรุง Test รวมถึงการใช้ Coverage Feedback เพื่อวิเคราะห์ชุดทดสอบที่สร้างโดย AI ใดก็ตาม แนวทางดังกล่าวยังไม่ได้ถูกทดลองในโครงการนี้ จึงควรถือเป็นข้อเสนอแนะสำหรับการศึกษาลำดับต่อไป และไม่ควรสรุปว่าเป็นผลที่ได้รับการยืนยันจากการทดลองครั้งนี้

บทที่ 6

ข้อจำกัด ปัญหา และความสามารถในการทำซ้ำการทดลอง

6.1 ข้อจำกัดของการทดลอง

การทดลองทั้ง 4 แนวทางมีข้อจำกัดหลายประการที่อาจส่งผลกระทบต่อความดีความและการเปรียบเทียบผลการทดลอง โดยมีรายละเอียดดังนี้

1. การสร้างชุดทดสอบสำหรับแต่ละ Bug ดำเนินการเพียงหนึ่งครั้ง โดยไม่มีการสร้าง Test ใหม่จาก Prompt เดิมหรือปรับปรุง Test ที่สร้างขึ้นด้วยวิธีการอื่น ดังนั้น ผลการทดลองจึงยังไม่สามารถสะท้อนความแปรปรวนของผลลัพธ์ที่อาจเกิดขึ้นจากการสร้างชุดทดสอบหลายครั้งได้
2. Claude AI Sonnet 5.5 และ Gemini 3.8 Flash ถูกนำไปทดลองกับชุด Project และ Bug ที่แตกต่างกัน เนื่องจากข้อจำกัดด้าน API Quota ส่งผลให้ค่า Fault Detection Rate ของทั้งสองเครื่องมือไม่สามารถนำมาเปรียบเทียบโดยตรงเพื่อสรุปประสิทธิภาพของโมเดลได้ โดยจำเป็นต้องพิจารณาความแตกต่างของชุดข้อมูลและเงื่อนไขการทดลองร่วมด้วย
3. Prompt ที่ใช้กับ Claude AI Sonnet 5.5 และ Gemini 3.8 Flash มีความแตกต่างกันบางส่วน โดย Gemini มีการเพิ่ม EFFICIENCY NOTE เพื่อควบคุมการใช้ Token และลดปัญหา Response ไม่สมบูรณ์จากกระบวนการ Thinking ขณะที่ Claude ไม่มีข้อความดังกล่าว ความแตกต่างของ Prompt จึงอาจมีผลต่อรูปแบบและคุณภาพของ Test Code ที่สร้างขึ้น
4. การวัด Test Coverage พิจารณาเฉพาะคลาสที่ได้รับการแก้ไขของ Bug บน Fixed Version ไม่ได้ครอบคลุม Source Code ทั้งหมดของโครงการ ดังนั้นค่า Coverage ที่รายงานจึงแสดงระดับความครอบคลุมของส่วนที่เกี่ยวข้องกับ Bug ที่นำมาทดลอง และไม่ควรถือความว่าเป็น Coverage ของทั้งโครงการ
5. ผลการวัด Coverage บางรายการมีค่า 0.0% ซึ่งในบางกรณีไม่สามารถแยกได้อย่างสมบูรณ์ว่าค่าดังกล่าวเกิดจาก Test ไม่สามารถเข้าถึง Source Code หรือเกิดจากไม่มีข้อมูลที่สามารถนำมาวัด Coverage ได้ ส่งผลให้การคำนวณค่าเฉลี่ย Coverage ของบางโครงการอาจได้รับผลกระทบจากกรณีดังกล่าว
6. การประเมินผลของ Mockito, JacksonCore และ Collections ใช้สคริปต์รุ่นแรก ขณะที่ Compress ใช้สคริปต์ gemini_eval2.sh ซึ่งได้รับการปรับปรุงให้สามารถบันทึกผลแยกตาม Bug และรวมผลการทดลองโดยอัตโนมัติ แม้เกณฑ์การพิจารณา Fault Detection จะเหมือนกัน แต่ยังไม่มีการทำ Regression Test เพื่อยืนยันว่าการทำงานของสคริปต์ทั้งสองรุ่นให้ผลลัพธ์เหมือนกันในทุกกรณี

7. ข้อจำกัดด้าน API Quota ส่งผลต่อจำนวน Bug และจำนวน Project ที่สามารถนำมาทดลองด้วย AI-Assisting Tools ทำให้ไม่สามารถดำเนินการทดลองได้ครบทุกโครงการใน Defects4J ตามขอบเขตที่กำหนดไว้
8. กรณีที่ไม่สามารถวัดผลได้และกรณี Response ไม่สมบูรณ์ถูกแยกออกจากตัวหารของ Fault Detection Rate หลัก เพื่อไม่ให้ปัจจัยจาก Environment และข้อจำกัดของ API ถูกนำมาปะปนกับความสามารถในการตรวจจับข้อบกพร่องของ Test อย่างไรก็ดีตาม การไม่นับกรณีดังกล่าวอาจทำให้ค่า FDR สูงกว่ากรณีที่น่าผลลัพธ์เหล่านี้นับเป็นความล้มเหลว จึงควรพิจารณาผลการวิเคราะห์ความไว้วางใจด้วย
9. ผลการทดลองบางกรณียังจำเป็นต้องตรวจสอบสาเหตุของความล้มเหลวเพิ่มเติม โดยเฉพาะกรณีที่ Test ล้มเหลวทั้ง Buggy Version และ Fixed Version หรือกรณีที่ Test Compile ไม่ผ่าน เพื่อแยกสาเหตุระหว่างข้อผิดพลาดของ Test Code กับปัญหาที่เกิดจาก Helper, Classpath หรือ Environment
10. ผลการทดลองของโครงการ Compress ยังไม่ครอบคลุม Bug ทั้งหมด เนื่องจาก Bug 46 และ Bug 47 ยังไม่ได้ดำเนินการ ดังนั้นผลที่นำเสนอสำหรับ Compress จึงเป็นผลจาก Bug ที่สามารถดำเนินการและวัดผลได้ในช่วงการทดลอง
11. การทดลอง Gemini 3.8 Flash ยังไม่ครอบคลุมโครงการ Chart, Lang, Jsoup, Math, JacksonDatabind และ Closure เนื่องจากข้อจำกัดด้าน API Quota และระยะเวลาในการดำเนินการ ดังนั้นผลการทดลองที่ได้จึงไม่สามารถใช้เป็นตัวแทนของความสามารถของ AI-Assisting Tools บนชุดข้อมูล Defects4J ทั้งหมดได้

6.2 ปัญหาและอุปสรรคในการดำเนินการทดลอง

6.2.1 ปัญหาด้าน Defects4J ในการดำเนินการทดลองกับ Defects4J เวอร์ชัน 3.0.1 พบว่าคำสั่งบางส่วนจาก Workflow ของ Defects4J รุ่นก่อนหน้านี้ไม่สามารถนำมาใช้ได้โดยตรง เช่น `defects4j gen_tests` ส่งผลให้การสร้างชุดทดสอบด้วย Whole Test Suite Generation จำเป็นต้องเรียกใช้ EvoSuite โดยตรง และพัฒนากระบวนการสำหรับจัดการ Checkout, Compile และ Classpath ให้สอดคล้องกับสภาพแวดล้อมของการทดลอง

6.2.2 ปัญหาด้าน Java Version การทดลองมีการใช้ Java ต่างเวอร์ชันในแต่ละส่วนของระบบ โดย API Client สำหรับเรียกใช้ AI-Assisting Tools ใช้ Java 17 ขณะที่ Defects4J และกระบวนการวัด Coverage ใช้ Java 11 ความแตกต่างดังกล่าวจำเป็นต้องควบคุม Java Version ของแต่ละขั้นตอนให้ถูกต้องเพื่อป้องกันปัญหาความเข้ากันได้ระหว่างเครื่องมือและโครงการที่นำมาทดลอง

6.2.3 ปัญหาด้าน API Quota การเรียกใช้ AI-Assisting Tools ผ่าน KCU IntelSphere API มีข้อจำกัดด้าน API Quota ซึ่งส่งผลโดยตรงต่อจำนวน Request และจำนวน Bug ที่สามารถนำมาทดลองได้ ทำให้ไม่สามารถดำเนินการทดลองกับทุก Project ใน Defects4J ได้ภายในเงื่อนไขของการทดลอง

6.2.4 ปัญหาด้าน Response ไม่สมบูรณ์ ในการทดลอง Gemini 3.8 Flash พบว่าบาง Response สิ้นสุดลงก่อนที่จะสร้าง Test Code ได้อย่างสมบูรณ์ เนื่องจาก Token ที่ใช้ในกระบวนการ Thinking รวมอยู่ภายใต้ข้อจำกัด max_tokens ส่งผลให้ Response สิ้นสุดด้วย stop_reason = max_tokens และไม่สามารถนำ Test Code ดังกล่าวไปใช้ในขั้นตอนการ Compile และประเมิน Fault Detection ได้

6.2.5 ปัญหาด้านการ Compile ของ Test พบว่า Test Code บางรายการที่สร้างโดย AI-Assisting Tools ไม่สามารถ Compile ได้ เนื่องจากการอ้างอิง Class, Method หรือ API ที่ไม่มีอยู่จริง หรือไม่สามารถใช้งานได้ ใน Environment ของโครงการ ปัญหาดังกล่าวส่งผลให้ Test ไม่สามารถเข้าสู่ขั้นตอนการ Execute และการประเมินความสามารถในการตรวจจับข้อบกพร่องได้

6.3 ความสามารถในการทำซ้ำการทดลอง

เพื่อสนับสนุนความสามารถในการตรวจสอบและทำซ้ำการทดลอง โครงการนี้ได้จัดเก็บองค์ประกอบที่เกี่ยวข้องกับกระบวนการทดลองอย่างเป็นระบบ ได้แก่ Source Code ของเครื่องมือ, Shell Script, Configuration, Prompt, Generated Test, Raw API Response, Result CSV, Coverage Result, Log และ README

สำหรับการทดลอง AI-Assisting Tools ได้จัดเก็บ Prompt แยกตาม Project และ Bug เพื่อให้สามารถตรวจสอบข้อมูลที่ใช้ในการสร้าง Test ของแต่ละ Bug ได้

นอกจาก Prompt แล้ว ยังมีการจัดเก็บ Response ที่ได้รับจาก API และ Test Code ที่สร้างขึ้น แยกตาม Project และ Bug เพื่อให้สามารถตรวจสอบย้อนกลับกระบวนการสร้างชุดทดสอบได้ ผลการทดลองถูกจัดเก็บในรูปแบบตารางข้อมูล โดยประกอบด้วยข้อมูลสำคัญ เช่น

- bug
- test_files
- methods
- compile_buggy
- test_buggy
- compile_fixed
- test_fixed
- fault_detected
- lines_covered
- lines_total

- line_pct
- cond_covered
- cond_total
- cond_pct
- note

ข้อมูลดังกล่าวช่วยให้สามารถตรวจสอบผลการทดลองของแต่ละ Bug ได้ตั้งแต่ขั้นตอนการ Compile การ Execute บน Buggy Version และ Fixed Version การพิจารณา Fault Detection ไปจนถึงการวัด Test Coverage นอกจากนี้ การจัดเก็บ Prompt, Response, Test Code และผลการประเมินแยกตาม Bug ยังช่วยสนับสนุนการตรวจสอบย้อนกลับและการทำซ้ำการทดลองภายใต้ Configuration เดียวกันได้อย่างเป็นระบบ

บทที่ 7

สรุปผลและข้อเสนอแนะ

7.1 สรุปผลการศึกษา

โครงการนี้ได้ศึกษาแนวทางการสร้างชุดทดสอบอัตโนมัติจำนวน 4 แนวทาง โดยแบ่งออกเป็น Automated Test Case Generation Algorithms จำนวน 2 แนวทาง ได้แก่ Whole Test Suite Generation (WTS) และ Coverage-guided Greybox Fuzzing และ AI-Assisting Tools จำนวน 2 แนวทาง ได้แก่ Claude Sonnet 5.5 และ Gemini 3.8 Flash โดยใช้ Defects4J เป็นชุดข้อมูลสำหรับประเมินความสามารถของชุดทดสอบในการตรวจจับข้อบกพร่องจริงของโครงการภาษา Java

ในส่วนของ Automated Test Case Generation Algorithms ได้ศึกษา Whole Test Suite Generation ผ่านเครื่องมือ EvoSuite ซึ่งใช้แนวคิด Search-Based Software Testing ในการค้นหาและปรับปรุง Test Suite โดยอาศัย Fitness และ Coverage เป็นข้อมูลสำหรับประเมินและปรับปรุงชุดทดสอบ นอกจากนี้ยังได้ศึกษา Coverage-guided Greybox Fuzzing ซึ่งใช้ Coverage Feedback เพื่อควบคุมกระบวนการสร้างและปรับเปลี่ยน Input สำหรับค้นหาเส้นทางการทำงานใหม่ของโปรแกรม

ในส่วนของ AI-Assisting Tools ได้ศึกษา Claude Sonnet 5.5 และ Gemini 3.8 Flash ผ่าน KIU IntelSphere API โดยใช้ความสามารถของ Generative AI ในการวิเคราะห์ข้อมูลของ Bug, Trigger Test และ Source Code Diff แล้วสร้าง Test Code ระดับ Unit Testing จาก Prompt ที่สร้างขึ้นตามข้อมูลของแต่ละ Bug จากนั้นนำ Test ที่สร้างขึ้นไปประเมินกับ Buggy Version และ Fixed Version เพื่อพิจารณาความสามารถในการตรวจจับข้อบกพร่อง

จากผลการทดลองของ WTS พบว่ามีค่า Line Coverage เฉลี่ย 74.50% ในรอบที่ 1 และ 64.60% ในรอบที่ 2 ขณะที่ Coverage-guided Greybox Fuzzing มีค่า Line Coverage เฉลี่ย 23.66% และ 23.62% ตามลำดับ อย่างไรก็ตาม ค่าเหล่านี้ไม่ควรนำมาใช้จัดอันดับประสิทธิภาพโดยตรง เนื่องจากวิธีการวัด Coverage และหน่วยของเวลาในการทดลองแตกต่างกัน รวมถึง Greybox Fuzzing มีบางรายการที่ไม่มีข้อมูล Coverage เพียงพอสำหรับการคำนวณ

สำหรับ AI-Assisting Tools ผลการทดลองของ Claude Sonnet 5.5 ครอบคลุม 9 โครงการ มี Bug ทั้งหมด 424 รายการ สามารถตรวจจับ Bug ได้ 306 รายการ และมี FDR รวมจาก Bug ที่สามารถวัดผลได้เท่ากับ 75.37% ขณะที่ Gemini 3.8 Flash ครอบคลุม 8 โครงการ มี Bug ทั้งหมด 430 รายการ สามารถตรวจจับ Bug ได้ 332 รายการ และมี FDR รวมเท่ากับ 79.05% โดย Claude มีค่าเฉลี่ย FDR รายโครงการ 85.50% สูงกว่า Gemini ที่มีค่าเฉลี่ย 79.84% และมีค่าเฉลี่ย Line Coverage และ Condition Coverage สูงกว่า Gemini

อย่างไรก็ตาม ผลดังกล่าวยังไม่สามารถใช้สรุปว่าแนวทางใดมีประสิทธิภาพสูงที่สุด เนื่องจากแต่ละแนวทางมีขอบเขต Project และ Bug วิธีการสร้าง Test หน่วยการวัด และเงื่อนไขการทดลองแตกต่างกัน โดยเฉพาะ Claude Sonnet 5.5 และ Gemini 3.8 Flash ที่ไม่ได้ทดลองกับชุด Project และ Bug เดียวกันทั้งหมด

ดังนั้น ผลการศึกษาที่นำเสนอในโครงการนี้ควรพิจารณาตามขอบเขตและเงื่อนไขของข้อมูลที่ได้จากการทดลอง โดยผลลัพธ์ที่ได้สามารถใช้แสดงแนวโน้มและลักษณะของแต่ละแนวทาง แต่ยังไม่ควรนำมาใช้จัดอันดับประสิทธิภาพของทั้ง 4 แนวทางโดยตรง

7.2 สรุปผลการเปรียบเทียบทั้ง 4 แนวทาง

ผลการศึกษาพบว่า การสร้างชุดทดสอบอัตโนมัติสามารถดำเนินการได้หลายแนวทาง โดยแต่ละแนวทางมีหลักการทำงานและจุดเด่นแตกต่างกัน Whole Test Suite Generation ใช้ Search-Based Software Testing เพื่อค้นหาและปรับปรุง Test Suite ตามเป้าหมายด้าน Fitness และ Coverage ขณะที่ Coverage-guided Greybox Fuzzing ใช้การสร้างและปรับเปลี่ยน Input ร่วมกับ Coverage Feedback เพื่อสำรวจเส้นทางการทำงานใหม่ของโปรแกรม

สำหรับ Claude Sonnet 5.5 และ Gemini 3.8 Flash ซึ่งเป็น AI-Assisting Tools สามารถใช้ความสามารถของ Generative AI ในการวิเคราะห์ Source Code และข้อมูลบริบทของ Bug เพื่อสร้าง Test Code จาก Prompt โดยไม่จำเป็นต้องกำหนด Test Case แต่ละรายการด้วยตนเอง

เมื่อพิจารณาลักษณะของกระบวนการพบว่า WTS และ Coverage-guided Greybox Fuzzing ใช้ข้อมูลจากการ Execute Program เป็น Feedback ในกระบวนการค้นหา Test หรือ Input ใหม่ ขณะที่ Claude Sonnet 5.5 และ Gemini 3.8 Flash ใช้ความสามารถของ Generative AI ในการวิเคราะห์ Source Code และข้อมูลบริบทของ Bug เพื่อสร้าง Test Code โดยตรง

แต่ละแนวทางจึงมีข้อดีและข้อจำกัดแตกต่างกัน WTS มีจุดเด่นด้านการค้นหาและปรับปรุง Test Suite ตามเป้าหมายที่กำหนด Coverage-guided Greybox Fuzzing มีจุดเด่นด้านการสำรวจ Input Space และค้นหาเส้นทางการทำงานใหม่ ส่วน AI-Assisting Tools มีความสามารถในการวิเคราะห์ Source Code และข้อมูลของ Bug เพื่อสร้าง Test Code โดยตรง อย่างไรก็ตาม Test ที่สร้างโดย AI อาจพบปัญหาด้านการ Compile, Assertion, Dependency และความสมบูรณ์ของ Response จากข้อมูลที่ได้จากการทดลองจึงยังไม่สามารถระบุได้ว่าแนวทางใดมีประสิทธิภาพสูงที่สุด เนื่องจากวิธีการวัดผลและเงื่อนไขของแต่ละแนวทางแตกต่างกัน รวมถึง Claude Sonnet 5.5 และ Gemini 3.8 Flash ยังไม่ได้ทดลองกับชุด Project และ Bug เดียวกันทั้งหมด ดังนั้นผลการเปรียบเทียบควรใช้เพื่อแสดงแนวโน้มของแต่ละแนวทางมากกว่าการจัดอันดับประสิทธิภาพโดยตรง

7.3 ข้อจำกัดที่มีผลต่อการสรุปผล

ข้อจำกัดด้าน API Quota เป็นปัจจัยสำคัญที่ส่งผลต่อจำนวน Bug และจำนวนโครงการที่สามารถนำมาทดลองด้วย AI-Assisting Tools ทำให้การทดลองของ Claude Sonnet 5.5 และ Gemini 3.8 Flash ยังไม่ครอบคลุมโครงการและ Bug ทั้งหมดตามขอบเขตที่กำหนดไว้

นอกจากนี้ ผลการทดลองของ AI-Assisting Tools ยังมีบางรายการที่ไม่สามารถนำมาประเมินผลได้ เนื่องจาก Test Code ไม่สามารถ Compile หรือ Execute ได้ หรือเกิดปัญหาจาก Response และ สภาพแวดล้อมในการทดลอง รวมถึงในบางกรณี Test อาจทำงานล้มเหลวทั้ง Buggy Version และ Fixed Version ทำให้ไม่สามารถระบุความสามารถในการตรวจจับ Bug ได้อย่างชัดเจน

ความแตกต่างของชุด Project และ Bug ที่ใช้กับ Claude Sonnet 5.5 และ Gemini 3.8 Flash รวมถึงความแตกต่างของ Prompt และเงื่อนไขในการทดลอง เป็นปัจจัยที่ต้องพิจารณาเมื่อวิเคราะห์ค่า Fault Detection Rate และ Code Coverage ดังนั้นค่า FDR และ Coverage ของทั้งสองเครื่องมือจึงไม่ควรถูกนำมาใช้เป็นหลักฐานเพียงอย่างเดียวในการสรุปประสิทธิภาพของเครื่องมือ

สำหรับการเปรียบเทียบ WTS และ Coverage-guided Greybox Fuzzing ยังมีข้อจำกัดจากความแตกต่างของหน่วยการวัดและวิธีการคำนวณ โดย WTS รายงานเวลาเป็นต่อคลาส ขณะที่ Greybox Fuzzing รายงานเวลาเป็นต่อรายการรัน และ Greybox Fuzzing มีบางรายการที่ไม่มีข้อมูล Coverage เพียงพอสำหรับการคำนวณ

ด้วยข้อจำกัดดังกล่าว ผลการทดลองที่ได้จึงยังไม่สามารถใช้เป็นตัวแทนของประสิทธิภาพของทั้ง 4 แนวทางภายใต้เงื่อนไขเดียวกัน และการเปรียบเทียบเชิงตัวเลขควรดำเนินการภายใต้ชุดข้อมูล วิธีการวัดผล และเงื่อนไขการทดลองที่สามารถเปรียบเทียบกันได้

7.4 ข้อเสนอแนะสำหรับการดำเนินงานต่อไป

1. ดำเนินการทดลอง Whole Test Suite Generation ให้ครอบคลุมชุด Bug ที่กำหนด และบันทึกค่า Branch Coverage, Line Coverage, Mutation Coverage, Fault Detection Rate และ ระยะเวลาในการสร้างชุดทดสอบ
2. ดำเนินการทดลอง Coverage-guided Greybox Fuzzing ตาม Configuration ที่กำหนด โดยใช้ Budget 30, 60 และ 120 วินาที และดำเนินการซ้ำ 5 ครั้งต่อ Configuration เพื่อคำนวณค่าเฉลี่ย และส่วนเบี่ยงเบนมาตรฐาน
3. ขยายการทดลอง AI-Assisting Tools ให้ครอบคลุมโครงการและ Bug ที่เหลือใน Defects4J เพื่อเพิ่มความครอบคลุมของข้อมูลสำหรับการวิเคราะห์ผล
4. ดำเนินการทดลองในรายการที่ยังไม่สามารถประเมินผลได้ เพื่อเพิ่มความสมบูรณ์ของข้อมูลและลดจำนวน Bug ที่ไม่สามารถนำมาคำนวณ FDR ได้

5. ทดลอง Claude Sonnet 5.5 และ Gemini 3.8 Flash กับชุด Project และ Bug เดียวกัน เพื่อให้สามารถเปรียบเทียบความสามารถในการตรวจจับข้อบกพร่องได้โดยตรง
6. ควบคุมโครงสร้างและเนื้อหาของ Prompt ของ Claude Sonnet 5.5 และ Gemini 3.8 Flash ให้มีเงื่อนไขใกล้เคียงกันมากขึ้น เพื่อควบคุมปัจจัยที่อาจส่งผลต่อความแตกต่างของผลการทดลอง
7. ตรวจสอบ Test Code ที่ Compile ไม่ผ่านเป็นรายกรณี เพื่อแยกข้อผิดพลาดที่เกิดจาก Test Code ออกจากปัญหาที่เกิดจาก Helper, Classpath หรือ Environment
8. ตรวจสอบและเปรียบเทียบผลจาก Coverage Script รุ่นเดิมและรุ่นที่ได้รับการปรับปรุง เพื่อยืนยันความสอดคล้องของกระบวนการวัดผลก่อนนำข้อมูลจากทั้งสองรุ่นมารวมกัน
9. จัดทำตารางเปรียบเทียบผลการทดลองของทั้ง 4 แนวทางเมื่อมีข้อมูลครบถ้วน โดยพิจารณา Fault Detection Rate, Test Coverage, จำนวน Test, ระยะเวลาในการสร้างชุดทดสอบ และความสามารถในการ Compile และ Execute
10. วิเคราะห์ความสัมพันธ์ระหว่าง Test Coverage และ Fault Detection Rate เพื่อประเมินคุณภาพของชุดทดสอบอย่างรอบด้าน และหลีกเลี่ยงการใช้ Coverage เพียงตัวเดียวในการสรุปประสิทธิภาพของแต่ละแนวทาง

เอกสารอ้างอิง

1. Fraser, G., & Arcuri, A. (2011). *Evolutionary Generation of Whole Test Suites*. International Conference on Quality Software.
2. Arcuri, A., & Fraser, G. (2014). *On the Effectiveness of Whole Test Suite Generation*. International Symposium on Search Based Software Engineering.
3. Fraser, G., & Arcuri, A. (2025). *A Retrospective on Whole Test Suite Generation: On the Role of SBST in the Age of LLMs*. IEEE Transactions on Software Engineering.
4. Just, R., Jalali, D., & Ernst, M. D. (2014). *Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs*. Proceedings of the International Symposium on Software Testing and Analysis.
5. Zalewski, M. *American Fuzzy Lop (AFL)*.
6. Padhye, R., et al. (2019). *JQF/Zest: Coverage-Guided, Semantic Fuzzing for Java*. International Symposium on Software Testing and Analysis.
7. Gay, G., Staats, M., Whalen, M., & Heimdahl, M. (2015). *The Risks of Coverage-Directed Test Case Generation*. IEEE Transactions on Software Engineering.
8. EvoSuite. *Automated Test Generation for Java*.
9. Defects4J. *A Database of Real Faults in Java Programs*.